Positioning and Pain Monitors

Replication specification, version 1.0

1 November 2026. Private. Rebuild every method on a production machine.

Start here

Next action: run the six steps below in order. Steps 1 to 3 take about 15 minutes. They prove the reference code works on your machine before you touch any feed.

1. **Install** the reference code. Section 1. About 10 minutes.
2. **Run the tests.** All 72 must pass. About 1 minute.
3. **Run the live reconciliation.** It rebuilds bank numbers from public data. About 5 minutes.
4. **Connect your data.** Section 2. Half a day if a market data feed already exists.
5. **Switch on methods.** Start with P1 and P2 in Section 4. Then add indicators from Section 5.
6. **Schedule the jobs.** Section 9. About an hour.

What each status means

[Verified]	My build matches a number printed in a source, or every cell of a source file. It says nothing about whether the signal makes money.
[Specified]	The rules are complete and tested. No published output exists to compare against.
[Partial]	The source leaves gaps. This document fills them with stated decisions. Output may differ from the source.
[Not replicable]	The source uses private data or hides the rules. Only its structure is given.

> **Reproduced does not mean it works**
> Section 6 tested the signals after their sources' samples ended. No predictive signal survived. Section 7 tested them on credit ETFs, with the same result. Use these methods as descriptions of positioning until new evidence says otherwise.

The method map

ID	Method, and the question it answers	Status
P1	Morgan Stanley Pain Trade Monitor. How much new open interest is offside?	[Specified]
P2	PortfolioStrat, corrected. What share of one trader group's lots is in pain?	[Specified]
P3	PortfolioStrat legacy code, defects kept. What did the old system print?	[Verified]
P4	Activity by trade price. Where did buying and selling happen?	[Verified]
P5	The owner's workbook reading of P1. Is the net new position above or below water?	[Verified]
I1	Net position as a share of open interest.	[Verified]
I2	Goldman sentiment index. Where is net position in its 3-year range?	[Verified]
I3	Deutsche Bank scaled position, −10 to +10 over 4 years.	[Verified]
I4	One-standard-deviation rule on net position.	[Partial]
I5	Goldman filtered position and residual.	[Verified]
I6	Goldman options-implied position indicator (OPI).	[Partial]
I7	Deutsche Bank FX Cookbook: CFTC continuation, CFTC reversal, COFFEE.	[Partial]
I8	JPMorgan CTA proxy from order imbalance.	[Partial]
I9	Citi FX positioning indicator.	[Not replicable]
I10	Morgan Stanley Bond Market Indicators.	[Partial]
I11	Yan, Chan and Peters long-memory count model.	[Specified]
I13	Deutsche Bank ETF Flow Risk-o-Meter (EFRoM). Is ETF flow risk-on or risk-off?	[Partial]
I12	Five mentions with no usable rules.	[Not replicable]

Three things to know first

1. **Do not port the old code as it stands.** It removes too many contracts and compares sizes with prices. P2 is the corrected method. Use P3 only to explain old output.
2. **The Eurodollar contract is gone.** Its last CFTC row is 13 June 2023. Run P1 on 3-month SOFR instead.
3. **Keep the source PDFs off firm machines.** Several carry an exclusive-use mark that names someone else. This document states methods in its own words and does not need those PDFs. Ask compliance before you move even this document.

Words used throughout

Open interest	Contracts still open at the end of a day.
Offside, in pain	A position that loses money at the current price. A long entered above the price is offside. A short entered below it is offside.
Lot	A block of contracts added on one date at one entry price.
Ledger	The list of lots in a window, with how much of each is still open.
VWAP	Volume-weighted average price. Here it means one day's value, unless stated.
DV01	Dollars gained or lost per contract when the yield moves one basis point.
CTD	Cheapest-to-deliver bond, the bond a short futures holder would choose to deliver. A Treasury contract's DV01 is the CTD's DV01 divided by its conversion factor.
Report date	The Tuesday whose positions a CFTC report describes.
Release time	When the CFTC publishes that report. Normally Friday, 15:30 New York time.
CTA	Commodity trading advisor. A fund that mostly follows trends.

Contents

1 Install and prove the build 4

2 Connect the data 4

2.1 Which data each method needs . . . 5
2.2 The CFTC feed . . . 5
2.3 Timing: never use a report before its release . . . 6
2.4 Market data . . . 6
2.5 Data checks to run before any method . . . 7
2.6 Tables to store . . . 7

3 The ledger: the engine behind every Pain method 7

3.1 The algorithm . . . 7
3.2 Conventions used by every method . . . 8

4 The Pain methods 9

4.1 P1 Morgan Stanley Pain Trade Monitor . . . 9
4.2 P2 PortfolioStrat method, corrected . . . 11
4.3 P3 PortfolioStrat legacy code, defects kept . . . 13
4.4 P4 Activity by trade price . . . 14
4.5 P5 The owner's workbook reading of P1 . . . 14

5 The positioning indicators 15

5.1 I1 Net position as a share of open interest . . . 16
5.2 I2 Goldman Sachs sentiment index . . . 16
5.3 I3 Deutsche Bank scaled positioning . . . 17
5.4 I4 One-standard-deviation rule . . . 18
5.5 I5 Goldman filtered position and residual . . . 18
5.6 I6 Goldman options-implied position indicator . . . 20
5.7 I7 Deutsche Bank FX Cookbook signals . . . 22
5.8 I8 JPMorgan CTA proxy . . . 23
5.9 I9 Citi FX positioning indicator . . . 24
5.10 I10 Morgan Stanley Bond Market Indicators . . . 24
5.11 I11 Yan, Chan and Peters long-memory count model . . . 25
5.12 I12 Mentions with no usable rules . . . 26
5.13 I13 Deutsche Bank ETF Flow Risk-o-Meter (EFRoM) . . . 26

6 Do the methods still work? 28

7 Credit ETFs 30

8 Signals and tests 32

8.1 S1: the natural gas Pain rule from the 2018 deck . . . 32
8.2 How to test any signal . . . 32

9 Run it every day 32

10 Accept the build 33

11 Where the sources disagree 33

A Reference code 35

B Contract list and source index 51

1 Install and prove the build

Next action: copy the spec folder to the production machine and run these commands. The machine needs Python 3.10 or later. It also needs outbound HTTPS to `publicreporting.cftc.gov` and `fred.stlouisfed.org`.

```
cd spec
python3 -m venv .venv
. .venv/bin/activate
pip install -e ".[test]"
pytest -q                           # expect: 65 passed
python scripts/reconcile_live.py    # writes data/reconcile.json
```

Compare `data/reconcile.json` with Table 6. If the numbers match, the machine, the feed and the code all agree with the sources.

What is in the folder

Path	Content
`posmon/ledger.py`	The lot ledger and the removal rules. Section 3.
`posmon/pain.py`	Methods P1 to P5.
`posmon/indicators.py`	Methods I1 to I7 and I10.
`posmon/cta.py`	Method I8.
`posmon/glgarma.py`	The deterministic parts of I11.
`posmon/data.py`	CFTC feed, field map, release times and data checks.
`posmon/stats.py`	Newey-West regression and hit rates.
`posmon/cli.py`	Command line for the production jobs.
`config/contracts.csv`	Contract list: CFTC code, report type and dollars per point.
`data/ty_daily.csv`, `data/ty_cftc.csv`	Test inputs taken from the 2018 workbook.
`tests/`	The 72 acceptance tests.
`scripts/dv01/`	Builds the weekly Treasury futures DV01 file.
`research/`	Replication, out-of-sample and credit ETF studies, each with a plan, scripts and results.
`scripts/`	Reconciliation, and the scripts that write this document's numbers and figures.

Computed results in the method sections come from `generated/numbers.tex`, which the tested code writes. Research results, and any figure marked as coming from a research run, come from `research/*/results.json`. Rerun those with each folder's `run_all.sh`. To rebuild the PDF, run `./build.sh`.

2 Connect the data

Next action: fill the table below for your contracts, starting with one rates future and one energy future.

2.1 Which data each method needs

Method	Weekly CFTC data	Other data
P1	None	Daily close, daily VWAP, open interest, multiplier, FX rate
P2, P3	One long or short field	Daily adjusted close
P4	Long and short series	Daily price
P5	None	Daily high, low, close, volume, open interest
I1, I2	Legacy report	For tests: yields or returns
I4	Legacy report, combined	Weekly TY DV01. For tests: 10-year yield
I3	Traders in Financial Futures (TFF) report	None
I5	Legacy and TFF reports, combined	Yields at 2, 5, 7, 10, 20 and 30 years, weekly contract DV01
I6	None	Daily option open interest, volume and delta, by strike
I7	Legacy report	FX returns. COFFEE also needs DTCC option volumes.
I8	None	Tick-level trades, daily futures prices
I10	None	Bond excess returns, curve slope, equity returns, data surprises
I11	Any CFTC position series	None

2.2 The CFTC feed

The CFTC runs a free public API. All six datasets below answered live on 1 November 2026. The URL is `https://publicreporting.cftc.gov/resource/<dataset>.json`.

Report	Futures only	Futures and options
Legacy (non-commercial, commercial)	`6dca-aqww`	`jun7-fc8e`
Disaggregated (money manager, producer, swap)	`72hh-3qpy`	`kh3c-gbw2`
TFF (dealer, asset manager, leveraged)	`gpe5-46if`	`yw9f-hn96`

```
python -m posmon.cli fetch --report tff --code 043602 --start 2006-01-01 --out store/tff_043602.csv
```

Field names

This document uses short canonical names. The API uses other spellings. Map them with this table, or use `COLUMNS` in `posmon/data.py`. Codes for all contracts are in Appendix B.

Report	Canonical name	API column
All	`open_interest`	`open_interest_all`
Legacy	`noncomm_long`, `noncomm_short`	`noncomm_positions_long_all`, `noncomm_positions_short_all`
TFF	`lev_money_long`, `lev_money_short`	`lev_money_positions_long`, `lev_money_positions_short`
TFF	`asset_mgr_long`, `asset_mgr_short`	`asset_mgr_positions_long`, `asset_mgr_positions_short`
TFF	`other_rept_long`, `other_rept_short`	`other_rept_positions_long`, `other_rept_positions_short`
Disaggregated	`money_mgr_long`	`m_money_positions_long_all`

Filter on `cftc_contract_market_code`, and sort on `report_date_as_yyyy_mm_dd`.

Follow these four rules.

1. **Key on the CFTC code, never the market name.** Code `043602` has had at least three names, most recently `UST 10Y NOTE`.
2. **Use futures only by default, and combined for I4 and I5.** The workbook, Deutsche Bank and the Goldman sentiment index match futures only. Deutsche Bank's 229K in TY matches futures only (229,119) and misses the combined report (209,940). Goldman's filtered charts match the combined report (99.98% against 98.36% on futures only).
3. **Keep the field map.** API spellings are not uniform. One column is misspelt as `noncomm_postions_spread_all`. Use `COLUMNS` in `posmon/data.py`.
4. **Do not copy the old loader.** It used Quandl's futures-and-options suffix `FO_ALL`, which contradicts rule 2.

2.3 Timing: never use a report before its release

A report describes Tuesday. The CFTC releases it on Friday at 15:30 New York time. Using it from Tuesday is look-ahead. The old code did this. Figure 1 shows the only safe order.



Figure 1. The weekly timing rule. Entry prices come from the Tuesday close. Knowledge starts on Friday. Deutsche Bank's FX Cookbook then trades on Monday, a delay of 4 business days.

Decisions. The source is silent, so this specification chooses.
History: use `publish_time()`, which assumes the normal Friday release. **Live:** store the time your job first saw each row and use that. Reports can arrive late, for example in a US government shutdown. A fixed rule would then leak information.

2.4 Market data

Open interest. Use one of two modes and record which one.

- **Aggregate mode** sums all expiries. Use it for bond, equity, FX and energy futures. On report dates, it should equal the CFTC figure. It matched on 23 of 25 dates in the workbook.
- **Per-expiry mode** treats each expiry as its own contract. Use it for rate strips such as SOFR. The Morgan Stanley note speaks of "individual contract charts", but it does not define a contract.

VWAP. Use the best source you have, and store which one you used.

1. Exchange trades: $\sum_j p_j v_j / \sum_j v_j$ over the session.
2. Intraday bars: the same sum over bars. Use `vwap_from_bars`.
3. Typical price: the mean of whichever of H, L and C exist, as the 2018 workbook used. This is a fallback only.

Rolls. A raw front-contract price jumps at each roll. That jump creates false entry prices and false price changes. Back-adjust so the latest price stays true and earlier prices shift by each roll gap:

$$P_t^{\text{adj}} = P_t + \sum_{r\,:\,r>t} g_r, \qquad g_r = P_r^{\text{new}} - P_r^{\text{old}}.$$

The old code took adjusted closes (its argument type is `ADJ_CLOSE_SERIES`). The 2018 workbook did not. Use `back_adjust`. Per-expiry mode needs no adjustment.

2.5 Data checks to run before any method

These checks caught real faults in the workbook. Stop the job if a check marked "stop" fails.

Check	Action	What the workbook showed
TFF and legacy identities: long side plus spreads equals open interest	stop	No failures on TY
A week missing from the CFTC series	stop	11 Nov 2014 is missing from the sheet. The test files keep this gap on purpose.
Duplicate key in `options_eod`: date, expiry, strike, type	stop	Not in the workbook
Close outside the high–low range	flag	69 of 125 rows
High equals low	flag	35 rows
High or low missing	flag	19 rows
Volume missing	flag	1 row
Volume collapses against its 60-day median	flag	27 rows under 200,000 around the September and December rolls

The volume faults matter. The sheet pairs total open interest with front-contract volume, and that volume dies before each roll. So never weight by front-contract volume alone.

2.6 Tables to store

Table	Columns
`contracts`	symbol, cftc_code, report_for_traders, asset_class, dollars_per_point, mode
`cftc_weekly`	report, basis, code, report_date, first_seen, then one column per canonical field
`market_daily`	symbol, date, close, adj_close, high, low, volume, open_interest, vwap, vwap_source
`options_eod`	date, root, expiry, strike, is_call, forward, implied_vol, abs_delta, open_interest, volume
`trades`	symbol, time, price, size

3 The ledger: the engine behind every Pain method

Build this first. P1, P2 and P4's matched view all run on it. A ledger answers one question: of the positions added inside this window, how much is still open, and at what entry price?

3.1 The algorithm

Inputs are one window of dates, a signed change c_t for each date, and an entry price e_t for each date. Replay the window from the oldest date.

1. Start with an empty list of lots.
2. If $c_t > 0$, add a lot with gross c_t, open c_t and entry e_t.
3. If $c_t < 0$, set the request $R = -c_t$ and the available total $A = \sum_i o_i$.
4. Remove $\min(R, A)$ from the open lots under the chosen rule.
5. Record $R - \min(R, A)$ as **unmatched**. It means the reduction hit positions older than the window.

6. Rebuild from empty for every as-of date. Do not update yesterday's ledger.

The removal rules

Rule	What it does	Source
`pro_rata`	Every lot loses the same share: $o_i \leftarrow o_i - \min(R, A)\, o_i / A$, with A fixed for the whole pass.	Morgan Stanley 2014, June 2015 emails
`fifo`	Oldest lots go first.	2018 deck, page 16
`low_price_first`	Lowest entry prices go first.	2018 function text

Use `pro_rata`. Two independent sources describe it, and it is the only rule argued for in writing. The June 2015 email thread's reason was that you know positions are closing, but not which ones. Any other split would be arbitrary. Figure 2 shows why this choice matters.

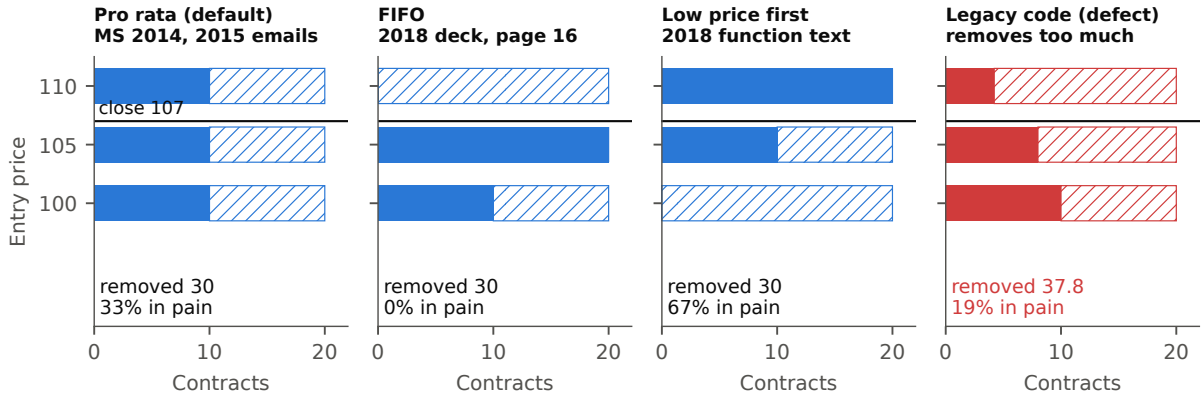


Figure 2. The removal rule changes the answer. All four panels start from the same market: 20 contracts added at 110, then 100, then 105, then a cut of 30. Solid is still open. Hatched is removed. The pain share is the open amount above the 107 close. It runs from 0% to 67% across the three stated rules. The legacy code also removes 37.79 instead of 30.

> **Check. Your build must give these numbers.**
> Lots 20, 20, 20 at 110, 100, 105, then a cut of 30. Pro rata leaves 10, 10, 10. FIFO leaves 0, 10, 20. Low price first leaves 20, 0, 10. Every rule removes exactly 30.

Three invariants. Check them on every run.

1. Each lot satisfies $0 \le o_i \le g_i$.
2. Gross minus removed equals open.
3. Requested equals removed plus unmatched.

`Ledger.check()` raises an error when any of them fails. The legacy code fails the second one.

3.2 Conventions used by every method

Use these unless a method says otherwise. They make two builds agree to many decimals.

Item	Convention
Rolling window	Includes today. Return no value until the window is full.
Standard deviation	Sample form, dividing by $n-1$.
Z-score	$(x_t - \text{mean})/\text{std}$ over the same window, today included.
Lengths	1, 3, 6 and 12 months are 21, 63, 126 and 252 trading days. A year of weekly data is 52 rows.
Half-life h	Weight $\alpha = 1 - 0.5^{1/h}$.
Span n	Weight $\alpha = 2/(n+1)$.
Exponential mean	$m_0 = x_0$, then $m_t = \alpha x_t + (1-\alpha)m_{t-1}$.
Exponential std	Use weights $w_0 = (1-\alpha)^t$ and $w_i = \alpha(1-\alpha)^{t-i}$. With $W_1 = \sum w_i$ and $W_2 = \sum w_i^2$, the variance is $\frac{W_1^2}{W_1^2 - W_2} \cdot \frac{\sum_i w_i (x_i - m_t)^2}{W_1}$.
Pandas equivalent	`ewm(halflife=h, adjust=False).mean()` and `.std()`.
Least squares	Include an intercept.

4 The Pain methods

4.1 P1 Morgan Stanley Pain Trade Monitor

[Specified] The source is a 16 October 2014 sales note. Its one page prints no output value, so nothing exists to match. P5 is the owner's simpler reading of the same idea.

What it answers. Of the positions opened in the last 30 trading days, how much is offside now?

Inputs, per contract

C_t close. V_t daily VWAP. O_t open interest. M multiplier. F_T USD per unit of quote currency on the as-of date T.

Steps

1. Compute $\Delta C_t = C_t - C_{t-1}$ and $\Delta O_t = O_t - O_{t-1}$.
2. Label each day with Table 1. All of the open-interest change goes to one side.
3. Take the last $N = 30$ daily changes. This needs 31 rows of levels.
4. Replay the long changes through the ledger, with entry V_t and the rule `pro_rata`.
5. Replay the short changes through a second ledger in the same way.
6. Set the test level to the as-of close C_T.
7. Compute the outputs below.
8. Rank contracts by Off$/O_T$, largest first, separately for longs and shorts.

Table 1. The four cases. The note states the idea. It does not print this table, but only this mapping fits its words.

	$\Delta O_t > 0$	$\Delta O_t < 0$
$\Delta C_t > 0$	New longs: add to longs	Shorts covered: remove from shorts
$\Delta C_t < 0$	New shorts: add to shorts	Longs taken off: remove from longs

Outputs, per side

$$\text{Off}_{\text{long}} = \sum_{i\,:\,e_i > C_T} o_i, \qquad \text{Off}_{\text{short}} = \sum_{i\,:\,e_i < C_T} o_i,$$

$$\text{share of new} = \frac{\text{Off}}{\sum_i o_i}, \qquad \text{share of open interest} = \frac{\text{Off}}{O_T}, \qquad \text{USD offside} = \sum_{i\ \text{offside}} o_i\, e_i\, M\, F_T.$$

Decisions. The source is silent, so this specification chooses.

- **Price change:** close to close. The note says only "price change".
- **No change:** if $\Delta C_t = 0$ or $\Delta O_t = 0$, make no entry and log the day.
- **Exact ties:** a lot entered exactly at C_T is not offside.
- **Denominators:** use contracts, not dollars, for the share of new. Report shares as fractions. If a side has no open lots, return no reading.
- **Older positions:** a reduction larger than the ledger is capped. Publish the excess as `unmatched`.
- **Rate strips:** the note's notional formula gives price times multiplier. For a SOFR contract that is far below its $1 million face. Rank on the share of open interest, which this does not affect.

Worked example: follow this by hand once

Use a seven-day window, $M = 1{,}000$ and $F_T = 1$.

Day	Close	VWAP	Open interest	ΔC	ΔO	Label
0	100.0	100.0	1,000			start row
1	101.0	100.6	1,100	+1.0	+100	new longs
2	102.0	101.6	1,160	+1.0	+60	new longs
3	101.2	101.7	1,210	−0.8	+50	new shorts
4	100.5	100.9	1,170	−0.7	−40	longs taken off
5	101.5	101.0	1,150	+1.0	−20	shorts covered
6	100.8	101.2	1,190	−0.7	+40	new shorts
7	101.3	101.1	1,180	+0.5	−10	shorts covered

Longs: days 1 and 2 add 100 at 100.6 and 60 at 101.6. Day 4 removes 40 from 160, a quarter of each lot. That leaves 75 and 45. Shorts: day 3 adds 50 at 101.7. Day 5 covers 20, leaving 30. Day 6 adds 40 at 101.2. Day 7 covers 10 from 70, one seventh of each, leaving 25.714 and 34.286.

Check. Your build must give these numbers.

Toy example at close 101.3. Long offside 45, which is 37.5% of new longs and 3.81% of open interest, or $4,572,000. Short offside 34.286, which is 57.1% of new shorts and 2.91% of open interest, or $3,469,714. Real TY data, 22 January 2015, typical price as the VWAP fallback, $N = 30$. Long open 90,152, offside 29.66%. Short open 9,996, offside 100.00%. Unmatched: 49,683 long and 28,730 short. These pin down the code. They are not market truth, because the workbook's high and low fields are unreliable.

Eurodollars

Code `132741` stops on 13 June 2023. Use `134741` (SOFR-3M) for live work. Nothing in the note says it is the document the thread refers to. It is the closest match found.

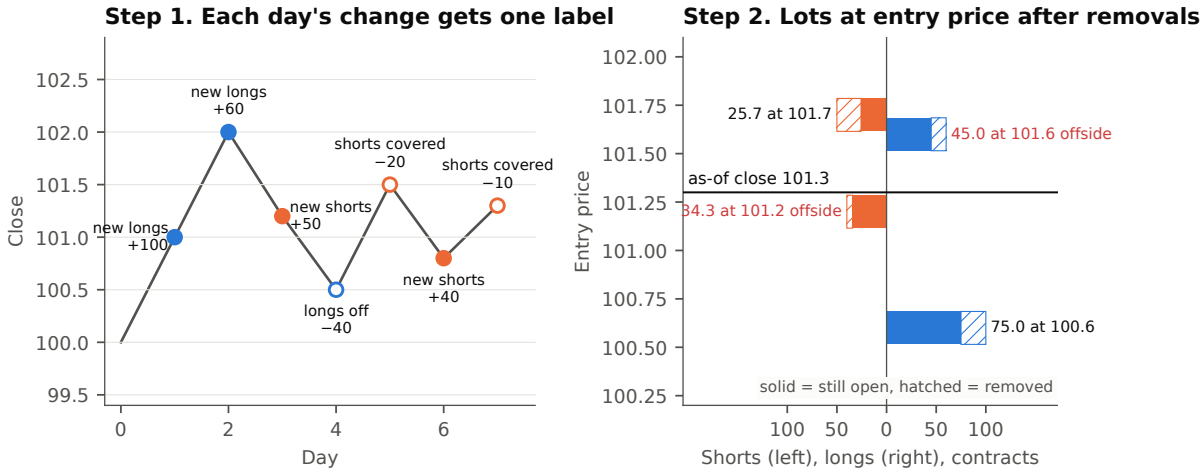


Figure 3. How P1 works. Left: filled markers add a position and open markers remove one. Blue is the long book. Orange is the short book. Right: the resulting lots. A long is offside above the close line. A short is offside below it. Red text marks the two offside lots.

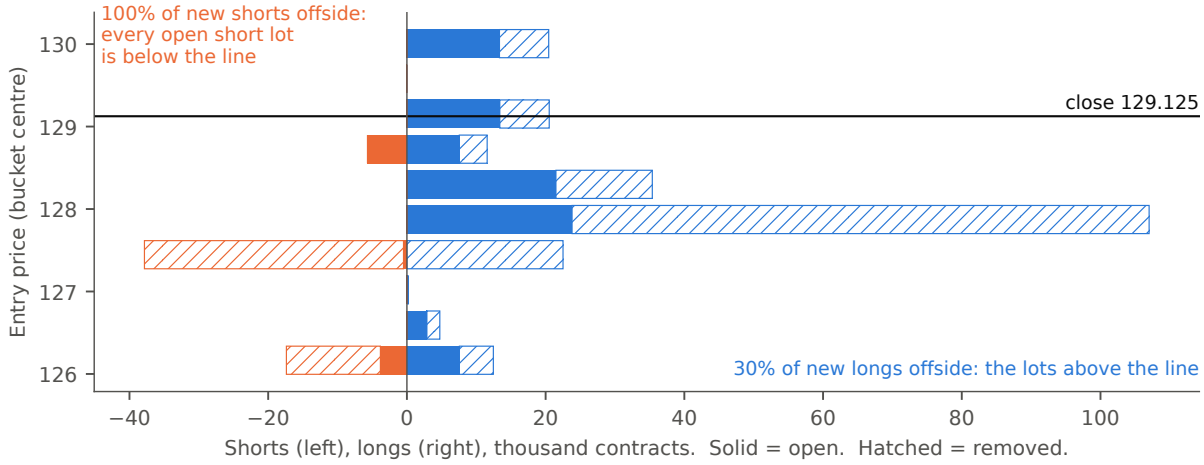


Figure 4. P1 on real TY data at 22 January 2015, grouped into ten price buckets for reading. This is the chart to reproduce on your screen. Compute the signal from lots, not from buckets.

4.2 P2 PortfolioStrat method, corrected

[Specified] Rules from the June 2015 emails, with the defects of P3 removed. No source output exists for the corrected formula. The deck's line is the legacy one, as P3 shows.

What it answers. For one trader group in one market, what share of the positions opened in the window sits at a losing entry price?

Why not P1? P1 infers the side from price and open interest. P2 reads the side directly from the CFTC, at the cost of weekly data.

Inputs

One weekly CFTC field, for example `lev_money_long` from TFF or `money_mgr_long` from the disaggregated report. The daily adjusted close. A window of N trading days.

Steps

1. Put the weekly field on the daily calendar. The change lands on its report date. All other days have zero change. If the report date is not a trading day, use the next trading day.
2. For each as-of date T, take the last N daily rows of change. Compute changes on the full series first.
3. Zero any change from a report not yet released on T. This is what removes the old look-ahead.
4. Replay them through the ledger. The entry price is the close on the change date. Use `pro_rata`.
5. Compute the share in pain with $L = C_T$:

$$\text{long field: } \frac{\sum_{i:e_i \geq L} o_i}{\sum_i o_i}, \qquad \text{short field: } \frac{\sum_{i:e_i \leq L} o_i}{\sum_i o_i}.$$

6. Build the ladder chart if you want one. For each ratio $r \in \{0.5, 0.6, \ldots, 1.5\}$, sum the open lots with $e_i \geq r\, C_T$. **The value must fall as r rises.**
7. Build the old bucket chart if you want one. Use ten equal buckets from the window's low to its high. Show gross, removed and open in each.

Choose the window

Rows	Where it comes from	Use
30	Function default and the old screen	Only to rebuild the 2015 screen. It holds about six reports.
90	Function description	Only to rebuild a saved workspace.
126	"Last six months" in the 2018 deck	**Default for signals.**

Decisions. The source is silent, so this specification chooses.

- **Short fields:** the sources show only long examples for the signal. Pain for a short is below the price, so the test flips.
- **Ties:** "at or above" follows the title of the deck's chart. So P2 counts a lot exactly at the price, and P1 does not. Keep this difference.
- **Gross** covers all lots in the window, not only those in pain.
- **Empty ledger:** when no lots remain open, return no reading. Do not return zero.
- **Weekly path:** a week's whole change lands on Tuesday's close. That is the sources' rule, and it hides the path inside the week.

Two outputs share one description

The deck says the output is the percent of open positions initiated above the price. The old `PCT_OPEN` code returns something else: open divided by gross, among lots at or above the price. That is a survival share. For a short field, the same test looks at lots at or below the price. Both are in the code as `share_in_pain` and `pct_open_code`. On TY leveraged-fund longs, with the release rule off, they read 26.18% and 100.00%. Trade the first. Use the second only to explain old output. The deck's chart is the second output. On 30 rows of combined-report data it stays inside the digitised line's band for 80% of dates, and reads the printed 1.00 on 30 January 2018. The first output reads 0.26 there.

Check. Your build must give these numbers.

TY `lev_money_long`, $N = 90$, as of 22 January 2015. With the release rule off (`respect_release=False`, to match old output): gross 147,591, open 52,912, unmatched 73,907, share in pain 26.18%. With the rule on, the share is 0.00%, because the 20 January report is not out until 23 January. That shows how much the rule matters. For `asset_mgr_long` with $N = 60$ and the rule off, no lots remain open, so the reading is empty.

4.3 P3 PortfolioStrat legacy code, defects kept

[Verified] Exact port of `functions.py`, lines 961 to 1050. Use it to explain old output, never for new signals.

The removal loop as written

Ten equal buckets run from the window's low to its high. A bucket's label is its lower edge. A price goes in bucket $\lfloor (p - \text{low})/\text{width} \rfloor$, and the top price goes in the last bucket. Each bucket keeps gross G_b and liquidated Q_b, and net starts at zero. An addition raises both G_b and net. Changes come from the full series, before the window is cut. Build each label as low $+\, b \times$ width, so exact ties behave the same. A reduction is ignored while net is zero. On a reduction of size x, the code walks the buckets from the lowest price:

$$\text{take}_b = \min\Big(\frac{G_b - Q_b}{\text{net}}\, x,\ G_b - Q_b \Big), \qquad Q_b \mathrel{+}= \text{take}_b, \qquad \text{net} \mathrel{-}= \text{take}_b.$$

Net shrinks inside the loop. So later buckets lose more, and the total passes x. With 20 in each of three buckets and $x = 30$, the takes are 10, then 12, then 15.79. The total is 37.79. The buckets lose 50%, 60% and 79%.

What the function returns at level L, for condition $>=$

`PCT_OPEN` is $\sum_{b:\text{label}_b \geq L} (G_b - Q_b)$ / $\sum_{b:\text{label}_b \geq L} G_b$, or zero if the bottom is zero. `OPEN` is the sum of $G_b - Q_b$ over buckets where that same open amount is at least L. `GROSS` and `CLOSED` work the same way on their own amounts. For `<=`, flip both tests. For `==`, it interpolates linearly across bucket labels. The function rolls over N daily rows and rebuilds each time.

Defects found

Defect	Evidence and effect
Removes too much	Net shrinks inside the loop. It removes 37.79 when asked for 30.
Compares size with price	Lines 1003 and 1009 test the bucket's quantity against the price level for `OPEN`, `GROSS` and `CLOSED`. See Figure 5.
`PCT_OPEN` is a survival share	It returns open over gross above the level. The deck describes a share of all open positions.
Three descriptions, none correct	The 2018 text says lower prices are liquidated fastest. The deck says FIFO. The code removes the largest share from the highest prices, by bucket order and not by date.
Buckets blur the test	A bucket is labelled by its lower edge. A lot above the level but inside its bucket is missed, an error of up to one bucket width.
Flat window fails	When the low equals the high, the bucket width is zero and the function fails.
Look-ahead	A change is used from its report date, three days before release.
Python 2 only	`xrange` and positional label access. The port in `pain.py` fixes these two syntax points and nothing else.

Check. Your build must give these numbers.
Changes $[20, 20, 20, -30, 0]$ at prices $[100, 105, 110, 102, 100]$ give removed 37.79. Changes $[10, 10, -5, 0]$ at $[100, 110, 105, 100]$ give removed 5.357. Natural gas ladder as of 30 January 2018, money-manager longs, 126 rows: 69,322 at every ratio. Outside check: the deck's line is this output. On 30 rows of combined-report data it stays inside the digitised line's band for 80% of dates and reads the printed 1.00. This comes from 50 variants, with one unfitted confirmation, on a coarse screenshot.

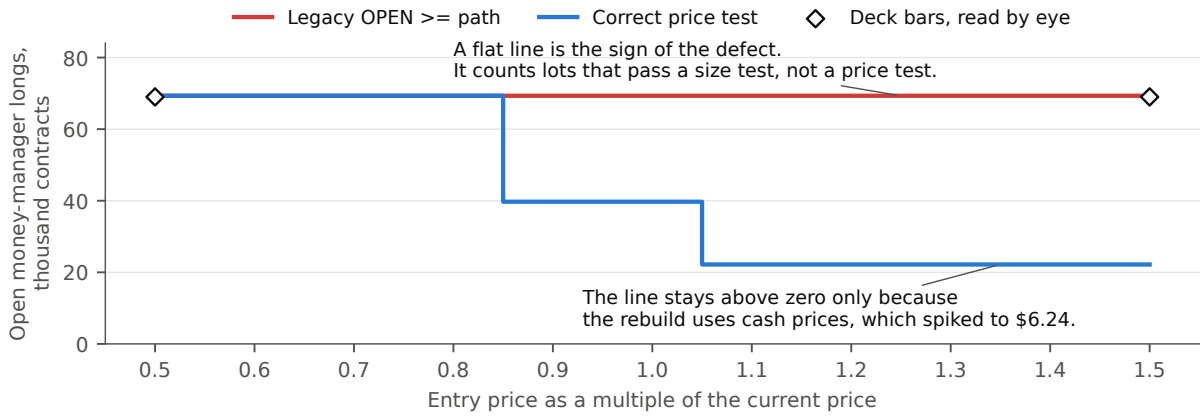


Figure 5. The natural gas ladder in the 2018 deck very likely carried the size-versus-price defect. Real CFTC money-manager longs through the legacy path give 69,322 contracts at every row. The deck's bars, read by eye, also sit flat near 69,000. A correct test must step down as the price multiple rises, and the blue line does. The rebuild uses Henry Hub cash prices because I have no futures history. The saved workspace is not in evidence, so this is strong evidence and not proof.

4.4 P4 Activity by trade price

[Verified] From the June 2015 email thread. This is not a Pain measure. The thread calls it an intermediate step.

What it answers. How much buying and selling happened at each price? It places every change at the price of the day it happened. It does not match exits to entries.

Steps

1. Compute the daily change of the long series and of the short series.
2. Split each into additions and removals, and group by that day's price.
3. Compute buys as longs added plus shorts taken off. Compute sells as shorts added plus longs taken off.

> **Check. Your build must give these numbers.**
> The thread's worked example: prices 100, 105, 110, 105, 110 with longs and shorts both 0, 6, 6, 3, 0. At 105: longs added 6, longs off 3, buys 9, sells 9. At 110: longs off 3, buys 3, sells 3. The entry-matched view of the same data is a ledger. It shows one lot at 105, gross 6, removed 6, open 0, which matches the Pain Monitor block of the Google Sheet.

4.5 P5 The owner's workbook reading of P1

[Verified] All seven formula columns match every cached cell, with a largest gap below 10^{-13}. Verified means the code matches the workbook. It does not verify P1 against Morgan Stanley.

What it answers. Is the market's net new position above or below water against a rolling average price? This workbook is the owner's own reading of the Morgan Stanley Pain idea. The file was last saved in January 2015 and emailed in 2018.

How it differs from P1

	P1	P5, the workbook
Positions	Separate long and short books at once	One net number, so one side at a time
Entry price	Each day's VWAP, for days that add positions	One blended price over 30 rows, including days when positions fell
Weights	The open interest added	All volume
Output	Contracts and dollars offside	A price gap only, with no size

On the workbook's 95 usable TY days, they agree on whether most of the position is under water on 73.7% of days. The correlation is 0.66. P1's larger book matches the workbook's side on only 48.4% of days, because P1 can hold large longs while net open interest falls. The workbook's side can also flip on a small change: it reads Short on 20 January 2015 and Long the next day. Rerun with `scripts/compare_p1_workbook.py`. This is one contract over about five months, so it shows how they differ, not which is right. Keep both. Neither has an out-of-sample test yet.

Columns, for each row t

$$
\begin{aligned}
G_t &= \text{mean of the available } H_t, L_t, C_t && \text{typical price}\\
J_t &= O_t - O_{t-1} && \text{change in open interest}\\
M_t &= \frac{\sum_{s=t-29}^{t} K_s G_s}{\sum_{s=t-29}^{t} K_s} && \text{called VWAP: 30 rows of volume } K\\
N_t &= \sum_{s=t-28}^{t} J_s && \text{net new position: 29 changes}\\
\text{Side}_t &= \text{Long if } N_t > 0, \text{ else Short}\\
P_t &= G_t - M_t && \text{over or under water}
\end{aligned}
$$

Both windows expand from the first row until they fill. A missing volume counts as zero. The sheet stops at P_t. I add the flag `underwater`: long with $P_t < 0$, or short with $P_t > 0$.

Faults in the workbook. Do not copy them.

- `M` is not a daily VWAP. It is a rolling weighted mean of typical price, so it differs from P1's V_t.
- `Sheet1` columns B and C use `VLOOKUP` on the wrong column, without an exact match, and with the wrong column number. Every row returns the last row's values.
- `Sheet1` column K adds dealer shorts and non-commercial longs to long fields from another report. It is not a total of longs.
- Column L holds an error value on 18 December 2014, because volume is missing there.

Check. Your build must give these numbers.

Last row, 22 January 2015: VWAP 128.698521, net new 47,968, side Long, over or under 0.238979.

5 The positioning indicators

Next action: build I1, I2 and I3 first. They need only the CFTC feed, and all three are verified. These methods measure how stretched positions are. None estimates entry prices.

5.1 I1 Net position as a share of open interest

[Verified] Deutsche Bank's weekly note, May 2017.

1. Take non-commercial longs and shorts from the legacy futures-only report.
2. Compute net = longs − shorts, and 100× net / open interest.
3. Add the 52-week high, low and median of that share. The note's first figure shows them.

Check. Your build must give these numbers.
Week to 9 May 2017, against the note's words. TY net 229,119 (229K), up 49,249 (49K). WN net −107,664 (−108K), change −18,902 (19K more short). FV change −45,087 (45K more short). US change −4,006 (pared 4K). Eurodollar change −89,554 (90K more short), share −23.6% (24%).

5.2 I2 Goldman Sachs sentiment index

[Verified] Sources: Global Viewpoint 12/17 (July 2012) and Global Markets Daily (18 July 2016).

What it answers. Where does today's net position sit between its 3-year low and high?

1. Compute net position NP_t in contracts from the legacy futures-only report.
2. Over the 156 weeks ending today, including today, find the low and the high.
3. Compute $SI_t = 100\,(NP_t - \text{low})\,/\,(\text{high} - \text{low})$.
4. Set $D_t^{90} = 1$ if $SI_t \geq 90$. Set $D_t^{10} = 1$ if $SI_t \leq 10$.
5. Add a counter of consecutive weeks in the current extreme. It captures how fresh the extreme is.

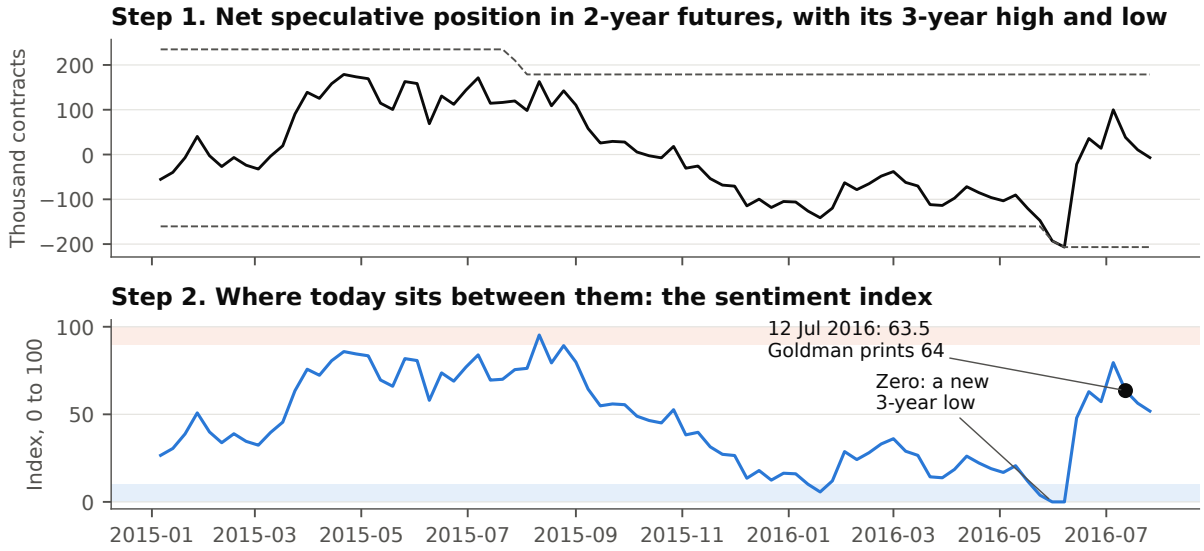


Figure 6. How to read the index. The top panel is the raw net position and its 3-year bounds. The bottom panel asks how far today sits between them. Shaded bands are the extreme zones. A reading of zero means a new 3-year low in net position.

The two regressions Goldman runs

$$R_{t+k} = \beta_0 + \beta_1 SI_t + \varepsilon_t, \qquad R_{t+k} = \beta_0 + \beta_1 SI_t + \beta_2 D_t^{90} + \beta_3 D_t^{10} + \varepsilon_t.$$

- **FX (2012):** R is the percent change in the currency against the dollar, for $k = 1, \ldots, 10$ weeks. The sample runs from January 1999 to July 2012.

- **Treasuries (2016):** R is the change in yield, for $k = 2, 4, 8$ weeks. The sample runs from December 2001 to June 2016. A positive β means a higher index goes with higher yields.
- **Standard errors:** Newey-West, because the returns overlap. Use `si_regression`.

Decisions. The source is silent, so this specification chooses.
The sources say "3-year" and give no lag count. Use 156 weeks. Set the Newey-West lag equal to k. Date a reading by the report date, then apply the release rule for any test.

Check. Your build must give these numbers.
EUR on 24 July 2012: 18.91 (Goldman prints 18.9% in its 31 July note). EUR zero readings in May and June 2012: 4 weeks (Goldman says four). 2-year on 12 July 2016: 63.5 (Goldman prints 64). 10-year on that date: 100.0 (Goldman calls it the longest in three years).
One gap: Goldman says the 2-year sat at zero for three weeks around end-May 2016. I get 2 weeks at zero. The week before reads 3.8.

Goldman's finding is worth keeping in mind. In 2016 extreme readings worked as a contrarian signal for 2-year and 5-year futures, but not for 10-year. In 2012, stretched EUR positioning led the currency for about a month instead of reversing it.

5.3 I3 Deutsche Bank scaled positioning

[Verified] Sales note, 15 May 2017. The note gives only the end points, so I tested three formulas against its chart.

What it answers. How large is today's net position against the largest long and largest short of the last four years?

1. Compute net contracts for one TFF group, futures only: leveraged funds or asset managers.
2. Over the last 209 weekly rows, which is four years counting both end dates, find the largest net long U and the largest net short W, as a positive number.
3. If net is positive, the score is $10 \times \text{net}/U$. If net is negative, it is $10 \times \text{net}/W$. If net is zero, the score is zero.

Check. Your build must give these numbers.
I read 22 values off the note's chart for 9 May 2017 and rebuilt them. Mean error 0.04 points, largest 0.13. A min-max scale gives a mean error of about 2.1, so it is wrong. Scaling by the largest absolute value gives 0.32.

Market	Leveraged, chart	Leveraged, rebuilt	Asset mgr, chart	Asset mgr, rebuilt
EUR	−3.0	−3.0	+10.0	+10.0
JPY	−0.3	−0.4	−5.7	−5.8
GBP	−1.2	−1.2	−6.5	−6.5
CHF	−2.3	−2.3	−1.5	−1.6
AUD	+2.7	+2.8	−0.5	−0.6
NZD	−4.5	−4.5	−3.9	−3.9
CAD	−8.2	−8.2	−5.2	−5.2
MXN	+6.5	+6.5	+5.4	+5.4
2Y	+2.0	+2.1	+6.8	+6.8
5Y	−4.8	−4.8	+6.8	+6.7
10Y	−2.1	−2.2	+3.9	+3.9

5.4 I4 One-standard-deviation rule

[Partial] Goldman, January 2019, as a benchmark inside its OPI report. The sample and base rate are likely pinned. The position recipe is not.

1. Sum net non-commercial contracts in TU, FV, TY and US from the legacy combined report. This is the closest recipe found, not Goldman's confirmed one.
2. Score it with the mean and standard deviation of June 2009 to January 2018.
3. A score of -1 or lower is a buy signal. A score of $+1$ or higher is a sell signal.
4. Measure hits against the 4-week change in Tuesday `DGS10`. Count a zero change as a rally.

Not matched

Goldman prints a 53% base rate, 63% buy and 48% sell. The sample above gives a base rate of 53.7%, and it fits the unconditional curve in Exhibit 1 (RMSE 0.025). No position recipe of 63,893 tried reproduces buy and sell inside that sample. This recipe gives 58.1% and 45.6%. Tuesday `DGS10` also spans less than the chart's range, so Goldman's yield series or timing differs. Treat this rule as a simple benchmark only.

Look-ahead

The frozen mean and standard deviation use the whole sample, so these hit rates are in-sample. After the sample, in a research run with a scale frozen on January 2007 to June 2018, the rule said buy in 282 of 386 weeks. Its buy hit rate was 43%, equal to the 43% base rate.

5.5 I5 Goldman filtered position and residual

[Verified] Global Rates Insights, 9 May 2019. This build tracks the weekly series drawn in Goldman's own charts at 99.98% and 99.91%.

What it answers. Have yields moved further than the positioning of the marginal investor implies? If so, expect some reversal over six months.

Steps

1. Use the combined futures-and-options report. Start in June 2006, and work weekly.
2. Compute net contracts by group and contract, using the table below. Add asset managers and other reportables before filtering.
3. For each contract, keep the weekly change in net only if it moves with that contract's yield. That means net selling as the yield rises, or net buying as it falls. Set the rest to zero.
4. Multiply each kept change by that week's contract DV01. Sum across contracts and divide by one million. The result f_t is in \$mm per basis point.
5. Regress the weekly change in `DGS10`, in basis points, on f_t. Use June 2006 to December 2010 only, with an intercept.
6. Compute the residual for all weeks: $\text{res}_t = \Delta y_t - \alpha - \beta f_t$.
7. Sum the residual over 26 weeks. Call the sum S_t.
8. Compute the score $Z_t = S_t / (\sigma\sqrt{26})$. Here σ is the standard deviation of the weekly residual over the fit window. Subtract no mean.
9. Score above $+0.25$: rally bias, long duration. Score below -0.25: sell-off bias, short duration. Otherwise, no view.

Group	Report and fields	Contracts
Asset managers and others	TFF: `asset_mgr` plus `other_rept`	TY, US
Leveraged funds	TFF: `lev_money`	TU, FV, TY, TN, US, WN
Specs	Legacy: `noncomm`	TU, FV, TY, TN, US, WN

Contract	TU	FV	TY	TN	US	WN
Yield that decides "with the market"	`DGS2`	`DGS5`	`DGS7`	`DGS10`	`DGS20`	`DGS30`

Decisions. The source is silent, so this specification chooses.

- **Late contracts:** TN and WN add nothing until they list.
- **Yield series:** use the last value on or before the report date.
- **Hit counts:** count weeks with a score beyond ± 0.25 and a known yield change 26 weeks later.
- **No look-ahead after the fit window:** α, β and σ stay frozen, so a live score uses only past data. The DV01 file is not point-in-time, but it moves slowly.

Build the DV01 file

Goldman's report says only "duration-weighted". No public table lists DV01 per contract, so `./scripts/dv01/refresh.sh` builds `data/treasury_futures_dv01.csv` from public data. It takes about 2 minutes. Rerun it monthly, when OFR updates.

1. **From March 2013:** the Office of Financial Research (OFR) publishes each contract's position in notional and in 10-year equivalents. Their ratio gives each contract's duration against TY. OFR's total DV01 over total 10-year equivalents gives the TY level. All eight group-and-side series give the same level, to the cent.
2. **Before March 2013:** build each contract's deliverable basket from TreasuryDirect records. Price each bond on the FRED yield curve. Compute each conversion factor with the CME formula. The CTD has the lowest price over conversion factor. DV01 is the CTD's DV01 over its conversion factor. Scale to meet OFR over its first 26 weeks.

The formula reproduces two published conversion factors exactly: 0.9506 and 0.8711. Over March 2013 to 2019, the bond build sits within 2 to 5% of OFR for FV, TY, TN, US and WN, and 9% for TU. It also catches the jump in US near the end of 2015, when the 2031 bond left the basket and the CTD became the 2036 bond.

How the report's hidden choices were found

Goldman's charts are vector graphics, so the PDF holds the weekly series itself. `scripts/goldman_chart_check.py` reads them and writes `data/goldman_chart_check.json`. It stores statistics only, never the series.

- **The charts fix four hidden choices.** They are the combined report, the filter yields, TY and US only for asset managers, and the score scale.
- **Weights confirm the build.** I undid the 26-week sums and fitted a weight per contract, with no DV01 input. Table 4 shows the fitted weights land on the real DV01 values. Asset managers load only on TY and US, in all three periods.
- **The score rule has no fitted number.** Goldman's implied scale is 63.4 for asset managers and 56.1 for specs. The rule gives 63.3 and 56.2.
- **Tracking.** Against Goldman's charts, the score correlations are 99.95% for asset managers and 99.95% for specs. The 6-month series reach 99.98% and 99.91%. Futures-only data gives 98.36% and 96.76%.

Table 3. I5 against the report, asset managers and others unless stated. Goldman quotes the first three rows as sizes on an inverted axis, so compare their sizes. My signs are negative because buying goes with falling yields.

Statistic	Goldman	This build
Yield change per $10mm/bp (bp), three groups	17 to 18	−17.2, −18.3, −18.1
6m filtered vs 6m yield change, asset managers	about 62%	−62%
6m filtered vs 6m yield change, leveraged funds	about 49%	−49%
Next-6m correlation, specs	−26%	−26%
Next-6m correlation, leveraged funds	−18%	−18%
Next-6m correlation, asset managers	−43%	−43%
Outright 6m change vs next 6m	−35%	−35%
Volatility of filtered changes, asset managers over specs	about 2/3	0.65
Size of ±0.25 score (bp)	about 15	15.8
Observations, base rally rate	620, 55.3%	622, 55.5%
Positive residual: count, hit rate, average (bp)	224, 74.1%, −40	228, 74.6%, −40.6
Negative residual: count, hit rate, average (bp)	253, 66.8%, +21	250, 66.8%, 21.4
Score on 30 April 2019	−0.67	−0.66

Table 4. Fitted weekly weight against real DV01, in dollars per contract. A dash means the contract was not listed.

Period	Series	TU	FV	TY	TN	US	WN
2006–2010	Real DV01	36	43	67	–	124	199
	Fitted, leveraged funds	36	43	68	–	111	207
	Fitted, asset managers	1	0	66	–	126	13
2011–2015	Real DV01	38	49	77	–	163	245
	Fitted, leveraged funds	37	46	78	–	156	253
	Fitted, asset managers	1	0	76	–	156	0
2016–2019	Real DV01	36	47	76	113	202	285
	Fitted, leveraged funds	36	47	78	110	195	272
	Fitted, asset managers	0	0	75	0	199	0

What this means. The method now reproduces. Two details remain unexplained: Goldman counts two fewer weeks, and the report gives no reason for dropping TU, FV, TN and WN from asset managers.

Do not trade I5

Verified means this build matches Goldman's charts. It does not mean the signal works. From May 2019 to October 2026, with Monday entry, the asset manager correlation with the next 26-week yield change is +4%. It was −27% for 2011 to April 2019 and −43% as published. The published number includes the fit period, where it reaches about −64%. Random flows through the same filter also give a slope near 18 bp per $10mm, because the filter keeps only flows that move with yields. See Section 6.

5.6 I6 Goldman options-implied position indicator

[Partial] Global Rates Insights, 17 January 2019. The bands, signals and hit rates reproduce from Goldman's own line. The raw OPI has not run on real markets, because I have no strike-level data.

What it answers. Are option trades building bullish or bearish exposure? Big rises in call open

interest against total volume read as bullish.

Steps

1. Each day, keep the front three option expiries.
2. Keep strikes with $|K - F| \leq 10$, where F is the forward itself. In 0.5 steps that is up to 41 strikes.
3. For each strike, get the daily change in open interest ΔOI, the volume V and the delta δ.
4. Compute the raw daily indicator:

$$OAI = \frac{\sum_c \delta_c \Delta OI_c - \sum_p \delta_p \Delta OI_p}{\sum_c \delta_c V_c + \sum_p \delta_p V_p}.$$

5. Smooth each of the four sums with a 25-day half-life, then rebuild the ratio:

$$OPI = \frac{EMA(\sum_c \delta_c \Delta OI_c) - EMA(\sum_p \delta_p \Delta OI_p)}{EMA(\sum_c \delta_c V_c) + EMA(\sum_p \delta_p V_p)}.$$

6. Compute bands on the OPI series. The centre is an exponentially weighted mean with a half-life of 63 days. The half-width is k times the standard deviation of all OPI values to date. This standard deviation expands. Use $k = 0.85$ for TY and $k = 1.25$ for Bunds.
7. Above the upper band, sell duration. Below the lower band, buy duration. Measure hits over the next 4 weeks.

Decisions. The source is silent, so this specification chooses.

- **Deltas are absolute values.** The report does not say so. But its stated bounds need it: -1 means every call trade is an unwind and every put trade is new. That gives -1 only with positive deltas.
- **Delta source:** Black-76 from settlement volatility, if your vendor has no delta. Use `black76_abs_delta`.
- **Bands use yesterday's values,** to avoid look-ahead. The report says "3-month half-lives". I use 63 trading days.
- **The band centre and width:** the report's "exponentially weighted standard error bands" describes only the centre. Its charts fix the width as an expanding standard deviation. An exponentially weighted width correlates only 0.906.

The report contradicts itself

Page 4 says a buy signal comes at the upper threshold. Its summary, its Exhibit 3 and its logic all say the opposite: a breach of the upper band is bullish positioning and a sell signal. This specification follows those three.

Check. Your build must give these numbers.

Formula tests: OAI equals $+1$ and -1 at the report's two stated limits. OAI stays within those bounds on 500 random strikes. Upper-band breach gives a sell.
Band recipe against Goldman's vector charts: centre 0.9997 and 0.9998, width 0.9994 and 0.9987. Implied k is 0.846 and 1.242, against printed 0.85 and 1.25. The Bund chart played no part in choosing the recipe.
From Goldman's drawn line and bands: sell and buy days 186 and 255, against 189 and 261. Hit rates 64.0% and 72.2%, against 64.0% and 73.2%. Combined 68.7%, against 69.3%. Likely recipe from 256 variants: start about April 2010, enter one day after the signal, and measure over 21 trading days. Means, medians, standard deviations and the 47% base rate were not in that search, and they match too.
Raw OAI and OPI remain untested. `marketdb` holds no CME options. Combined minus futures-only is not a stand-in: it never beats a correlation of 0.30 across 32 variants.

5.7 I7 Deutsche Bank FX Cookbook signals

[Partial] Quantcraft, 15 January 2019, sections 4.5 and 4.6. The rebuilt portfolios track Figure 42 at monthly correlations of 0.93, 0.82 and 0.80, with nothing fitted but the date map. COFFEE is not rebuilt, because it needs DTCC data.

CFTC continuation signal

With l_t and s_t the non-commercial longs and shorts,

$$s^{\text{cont}} = \frac{L - S}{L + S}, \qquad L = \sum_{h=0}^{3} l_{t-h}, \qquad S = \sum_{h=0}^{3} s_{t-h}.$$

CFTC reversal signal

1. Compute the z-score of the current net position over 1, 2 and 3 months.
2. Average the three z-scores.
3. Divide by the standard deviation of daily returns over the last 6 months. Do not annualise it.
4. Multiply by -1. Apply no smoothing.

COFFEE signal (Categorised Option Flow in Foreign Exchange)

1. Each day, take DTCC-reported European options expiring in under one year, excluding those expiring today.
2. Keep absolute deltas between 0.25 and 0.75.
3. Sum call notional minus put notional over the last 4 weeks.
4. Divide by the standard deviation of those 4-week sums over the last year.

Portfolios

Time series: $w_i = \text{sign}(s_i)\,(1/\sigma_i)/\sum_j(1/\sigma_j)$. Cross section: rank the signals, go long the top half and short the bottom half, with equal weights that sum to 100% in absolute value. In an odd pool, the middle asset gets zero. The report finds that the reversal signal works only in the cross section.

Decisions. The source is silent, so this specification chooses.

- **Lengths:** 1, 2 and 3 months are 4, 8 and 13 reports. COFFEE's 4 weeks are 20 trading days. Its volatility window is 252 trading days. Reversal volatility is 126 trading days of daily returns, read at the latest day on or before the report.
- **Timing:** trade on the Monday after release, as the report advises.
- **Currency sign:** CFTC contracts are long the foreign currency. Rank the cross section on signals that are long the foreign currency. Flip only to express the trade as USD/XXX, for USD/CAD, USD/JPY, USD/CHF, USD/BRL and USD/MXN. Ranking after the flip breaks the match.
- **Ruble:** keep USD/RUB until the CFTC series ends on 15 March 2022, then drop it. H.10 does not carry it. Use the Bank of Russia official rate.
- **Volatility in time-series weights:** 21 trading days of daily returns.
- **Windows:** a window of N weeks holds the mean of the last N weekly portfolios.
- **Returns:** spot plus carry. Spot alone keeps the correlation but cuts the time-series Sharpe from 0.29 to 0.12.

Check. Your build must give these numbers.

Rebuild from June 1999 to 2018, 10 currencies, weekly, with carry (`research/replication/i7_rebuild.py`). Correlation with Figure 42 at weekly, monthly and quarterly frequency: time-series continuation 0.84, 0.93, 0.94. Cross-sectional continuation 0.73, 0.82, 0.86. Cross-sectional reversal 0.69, 0.80, 0.81.

Rebuilt gross Sharpe 0.29, 0.22, 0.41, against chart net 0.35, 0.30, 0.26. Reversal turnover is 48, 33, 21 and 7.5, against printed 56, 42, 26 and 9.4.

Why I7 is not Verified, and two faults in the source

Weekly correlation stops at 0.84. The reversal Sharpe sits well above the chart's. Before June 1999 the fit is poor, because FRED no longer serves daily Deutsche mark rates. In the source, Figure 40's cross-sectional turnover line equals Figure 41's reversal line point for point. Figure 42 prints a reversal Sharpe of 0.25, while its text and bar say 0.22.

5.8 I8 JPMorgan CTA proxy

[Partial] Cross Asset CTA Activity Update, 30 October 2023. It points to an earlier report that is not in evidence. Choices below are mine.

What it answers. Are trades at CTA execution times following the trend signal? If they are, CTAs are probably adding. If they trade against it, they are probably unwinding.

Part 1: the trend signal (Baz et al. 2015, as restated by Lim et al. 2019)

1. For each pair $(S, L) \in \{(8, 24), (16, 48), (32, 96)\}$, compute exponential averages of price. A time-scale n has a half-life of $\log(0.5)/\log(1 - 1/n)$.
2. Set $q_t = (m_S - m_L)/\text{std}(P; 63 \text{ days})$.
3. Set $Y_t = q_t/\text{std}(q; 252 \text{ days})$.
4. Set the response $\phi(Y) = Y \exp(-Y^2/4)/0.89$. It peaks near $Y = \sqrt{2}$, then fades.
5. Average the three responses.

Part 2: the proxy

1. Classify each trade as buy or sell, and sum signed contracts per minute.
2. Choose the minutes that represent CTA execution, and sum imbalance across them each day.
3. Over a rolling month, compute the correlation and the beta of imbalance on the signal.
4. Score the beta against its own history with a zero mean: $z_t = \beta_t / \sqrt{\overline{\beta^2}}$, where the average covers all betas up to today.

Decisions. The source is silent, so this specification chooses.

- **Trade classifier:** the tick rule. The report does not name one.
- **Execution minutes:** unknown. Use `pick_execution_minutes` on past data, then freeze the list.
- **Averaging:** Lim et al. average trend strengths in their equation but speak of averaging positions. I average positions. Check Baz et al. before trading. I could not download it.
- **Month:** 21 trading days. Beta is the covariance over the variance of the signal.
- **Signal size:** the report's signals pass 1.5, so they cannot be an average of the response, which caps at 0.96. A sum of the three responses fits better on yield proxies, with a mean gap of 0.31 against 1.10. This is a likely recipe from 16 variants. Check it against futures prices.
- **Exponential averages:** seed with the first price, as in Section 3.2.

Check. Your build must give these numbers.

Formula tests only: ϕ peaks at $\sqrt{2}$, the beta recovers a known slope, and an uptrend gives a positive signal. The report's table to sanity-check against, 30 October 2023: TY beta 77,517 contracts, z-score 1.71, signal -1.78.

5.9 I9 Citi FX positioning indicator

[Not replicable] CitiFX Flows & Positioning, 29 April 2019. It rests on Citi's private client flows.

The indicator runs from −10 to +10 per currency. Its parts are below.

Block	Inputs named in the report
Leveraged	CitiFX flows, CFTC, CPAIN
Real money	CitiFX flows, CFTC
Retail	OANDA, ETFs, Japan retail

The weights are hidden. Simple averages do not reproduce the page. USD leveraged prints 0, but its parts (1, −2, −3) average −1.33. The report does not define CPAIN. A public proxy can use I3 on CFTC data. Do not label that proxy as Citi's indicator.

5.10 I10 Morgan Stanley Bond Market Indicators

[Partial] Global Rates Strategy, 6 March 2015. This is a duration model, not a positioning measure. The combination rules are verified. Two inputs are not fully stated.

The four factors, for each market

Factor	Rule
Carry	2s10s slope divided by 1-year realised volatility, as an expanding z-score from 1992.
Momentum	Excess-return index at $t-1$, less an exponential average of the prior index over 4 months. Divide by an exponential average of absolute changes over 4 months. No z-score. Scale it to match the other factors' volatility.
Equities	Average of two expanding z-scores: the 3-month local equity return and the EM minus DM return. Flip the sign, since weak equities favour bonds.
Business cycle	A composite of outcome minus median consensus across activity data. No z-score. Flip the sign.

Steps

1. Cap each factor to −10 to +10 with a cumulative distribution function. The printed factor tables are already capped, so do not cap them again.
2. Average the four into the combined signal.
3. Set it to zero if it lies between −1.5 and +1.5. Exactly ±1.5 also becomes zero.
4. Keep a market's signal only if at least one other market has the same sign. Otherwise it is zero.
5. Score on Thursday data. Trade on Friday morning. Hold until the next Thursday's close.
6. For a cross-market view, subtract one market's overall signal from another's, if at least one is live.

Decisions. The source is silent, so this specification chooses.

The report names "a cumulative distribution function" without its form. Use $10\,(2\Phi(z)-1)$. Expanding z-scores start after 52 weeks. The 4-month averages use a span of 17 weeks. With I the index, the numerator is I_{t-1} minus an average of I ending at $t-2$. The denominator is an average of $|\Delta I|$ ending at $t-1$. The equity inputs are 3-month returns, so compute those first. The report does not list the surprise series, so choose them and freeze them. The momentum and business-cycle scales are free constants, set to 1 in the code. Fit them on past data only.

The momentum span is unproven

The 17-week span gives the wrong sign for UST momentum on 5 March 2015 with a yield-built index: +0.99 against the printed -4.6. The sign turns negative only at spans of 9 weeks or less. My index is a proxy, so the proxy may be at fault.

Check. Your build must give these numbers.

Exhibit 1 of the report. Factors for UST, Bund, Gilt and JGB rebuild the printed combined signals 1.8, -0.8, 0.1 and -4.9. They also rebuild the printed overall signals, all 0.0. UST and JGB go to zero only because no other market shares their sign. That confirms step 4. Exhibit 2's vector lines give 632 values. Of 342 live values, 1 is inside the band and none lacks a same-sign partner.

5.11 I11 Yan, Chan and Peters long-memory count model

[Specified] SSRN 2962341. The deterministic parts are tested. I did not run the sampler. This is a forecasting model for CFTC position counts, not a Pain or reversal signal.

The model

$$Y_t \mid \mathcal{F}_{t-1} \sim D(\mu_t, \nu), \qquad (1 - 2uB + B^2)^d \, \phi(B) \, (\ln \mu_t - c) = \theta(B) \, \varepsilon_t.$$

B is the backshift operator. The error ε_t takes one of two forms. Parameter-driven: $\varepsilon_t \sim N(0, \sigma^2)$. Observation-driven: $\varepsilon_t = (Y_{t-1} - \mu_{t-1}) / \sqrt{\mu_{t-1}}$. Setting $u = 1$ gives the ARFIMA special case. The moving-average weights follow a recursion from $\psi_0 = 1$ and $\psi_1 = 2du$:

$$\psi_j = 2u\Big(\frac{d-1}{j} + 1\Big)\psi_{j-1} - \Big(\frac{2(d-1)}{j} + 1\Big)\psi_{j-2}.$$

The GP mass function, with mean μ and variance $\mu/(1-\nu)^2$, is

$$f(y; \mu, \nu) = \frac{\mu(1-\nu)\,[\mu(1-\nu) + \nu y]^{y-1}\, e^{-\mu(1-\nu) - \nu y}}{y!}.$$

Start-up (my choice): the first error is zero, $\ln \mu_0 = c$, and the first observation counts in the likelihood.

Steps

1. Take a weekly CFTC series. The paper uses 136 Treasury series from 13 June 2006 to 20 January 2015, 450 weeks, scaled to thousands or ten thousands of contracts.
2. Fit 16 base models: Poisson, negative binomial, generalised Poisson (GP) or double Poisson, by two error types, by ARFIMA or GARMA.
3. For GP, the paper's best distribution, add 24 models with AR and MA orders (0,1), (1,0), (1,1), (1,2), (2,1) and (2,2).
4. Use these priors: $u \sim U(-1, 1)$, $d \sim U(0, 0.5)$, $c, \phi_j, \theta_j \sim N(0, 10)$ where 10 is the variance, $\sigma^2 \sim \Gamma(3, 1)$. For ν: uniform on $(-1, 1)$ for GP, and gamma for negative binomial and double Poisson. I found no printed values for those two gamma priors, so set your own.
5. Sample with Stan's Hamiltonian Monte Carlo. The paper keeps 45,000 draws after burn-in. It truncates the infinite sum but gives no one number, so I use 100 terms. The observation-driven recursion can explode for a poor draw. If $|\ln \mu_t|$ passes 30, return a log likelihood of minus infinity so the sampler rejects that draw.
6. Choose models by the Deviance Information Criterion. Check effective sample size, Monte Carlo error and $\hat{R}$.

7. Forecast 45 weeks, one step ahead each time. Score with MAE, RMSE, MAPE in percent, and MASE. MASE divides each error by the mean absolute one-step change of the actual values in the forecast block.

Start with the parameter-driven form

The observation-driven form, as printed, puts a weight of one on the latest error. In my tests it explodes on short series, even with counts near their mean. I have not run the paper's data, so I cannot say how its fits avoided this. Test any fit by checking that the mean path stays bounded.

Check. Your build must give these numbers.

Tested: ψ_1 and ψ_2 match the paper's closed forms. With $u = 1$ they match ARFIMA. The GP mass function sums to one with the right mean. Use `od_filter` to test any Stan program you write.

5.12 I12 Mentions with no usable rules

[Not replicable] These are listed so you do not search for them again.

Item	What the source gives	What is missing
Deutsche Bank equity composite, Feb 2021	A weighted average of z-scores for CTA, risk-parity, vol-control and hedge-fund betas, fund flows, surveys, short interest, futures and options data	The weights ("based on explanatory power") and most beta definitions
Deutsche Bank CTA partial beta, 2017	CTA returns regressed on Treasuries, read as CTA positioning	Index, regressors, window
Deutsche Bank bond fund beta, 2017	Excess-return beta plus a weekly survey	Index, window, survey data
Morgan Stanley CTA monitor, 2016	One reading: a 1.30 z-score shift in WTI	All of the method
Robe and Roberts, 2019	Who holds agricultural futures	It uses non-public trader-level CFTC data. It gives no signal.

5.13 I13 Deutsche Bank ETF Flow Risk-o-Meter (EFRoM)

[Partial] US ETF Insights, 5 May 2016, with TAA from ETF Flows (14 May 2014), The Flow Whisperer (5 January 2015) and a June 2016 follow-up. The formula rebuilds one printed month with nothing fitted. Membership cannot be rebuilt exactly, because it is proprietary.

What it answers. Is ETF money trending into risky or safe assets? EFRoM is a risk-weighted sum of 14 flow-trend signals. Above zero means risk-on, and below zero means risk-off. The level describes the same month. The monthly change is the signal for next month.

Each step carries a label. **Stated** means the source says it. **Inferred** means the source implies it, and the check supports it. **Proposal** means the source is silent, and the choice is mine.

Inputs

- Daily shares outstanding and NAV for each ETF. Bloomberg in the source. (Stated)
- US-listed, long-only, non-levered ETFs and ETVs. No ETNs. SPY, IWM and the nine Select Sector SPDRs leave the sample. A new exposure needs 1 year and $500 million. (Stated)
- Segment membership, which is proprietary. Only one example ETF per segment is printed. (Stated)

Segment	Risk factor	Segment	Risk factor
US Treasuries	−1	US Large Cap	+0.5
Gold	−1	Broad Commodities	+1
Corp IG	−0.5	US Domestic Cyclicals	+1
Senior Loans	+0.5	DM International	+1
Corp HY	+0.5	US Global Cyclicals	+1
EM Debt	+0.5	US Small Cap	+1.5
US Defensives	+0.5	EM	+1.5

The factors are hand-set. The source explains the logic but gives no derivation. Defensives are staples, healthcare, telecom and utilities. Domestic cyclicals are industrials, consumer discretionary, financials and technology. Global cyclicals are materials and energy.

Steps

1. Compute each ETF's daily flow: $CF_t = (SO_t - SO_{t-1}) \times NAV_t$. (Stated.) Shares carry a 1-day lag in practice. (Stated.) Split-adjust, treat a missing day as zero flow, and date the flow by the data date. (Proposal)
2. Apply the universe rule and assign each ETF to a segment. (Stated for names, proposal for membership. Classify by index and strategy, then compare totals with the source's AUM snapshots.)
3. Sum flows across each segment's ETFs for each day. (Stated)
4. Use one calendar month, with cumulative flow at zero on the prior month-end. (Inferred)
5. Compute $Y_t = 100\times$ cumulative segment flow over segment AUM. (Stated.) Use AUM on the anchor date, held fixed for the month. (Proposal)
6. Fit a slope through the origin, with $x = 1, \ldots, n$ trading days: $b = \sum xY / \sum x^2$. (Stated for the regression, inferred for trading days. Calendar days give 0.43 against the printed 0.64.)
7. Compute R^2 from an ordinary regression with an intercept, on the n points. (Stated for the formula, inferred for leaving out the anchor.)
8. Compute $TAARSS_i = b_i \times R_i^2$, in percent of AUM per day. Use `taarss`. (Stated)
9. Compute $EFRoM_T = \sum_i TAARSS_i \times \text{RiskFactor}_i$. Use `efrom_level`. (Stated for the sum, inferred for raw values. The printed z-scores give −0.29.)
10. Compute $\text{Change}_T = EFRoM_T - EFRoM_{T-1}$. (Stated in words.)
11. If the change is positive, hold 100% global equities (ACWI) next month. If negative, hold 100% US Treasuries (IEI). (Stated.) If exactly zero, hold the previous position. (Proposal)

Do not add z-scores, thresholds, smoothing or an anomaly filter. No source states any of them for EFRoM.

> **Timing**
> The source's allocations flip at month-ends, yet month-end share data arrives the next morning. So its backtest appears to carry at least one day of look-ahead. Trade at the first close after the data is

known, and report results both gross and net of costs. (Proposal)

Check. Your build must give these numbers.
December 2014, rebuilt from the flow paths drawn in The Flow Whisperer, gives 0.638. The chart prints 0.64. Nothing is fitted. Slope alone gives 0.671, and an ordinary slope gives 0.719, so the check separates the formula from its near neighbours. On the same PDFs, weights for January 2015 reproduce to within 0.1 point. This is one month. It tests the formula on Deutsche Bank's own normalised paths, not membership or the AUM denominator. Scripts are in `research/efrom/`.

The evidence for the signal is thin
The source's test has 16 months. It reports a return of over 21%, against flat for ACWI. Next-month return on the prior change has R^2 0.06, and my reading of its t-statistic is about 1.0. The sign matches in 11 of 16 months. The source itself cautions about the small sample. I have run no out-of-sample test.

Credit. The index already holds Corp IG, Senior Loans, Corp HY and EM Debt, with LQD, BKLN, HYG and EMB as examples. They added −0.36 to the December 2014 level. My view, untested: a credit-only version can run steps 1 to 8 on pooled credit segments, with Treasuries as the safe partner, and test the change on a trade such as HYG against IEF. Any factor set is a new assumption. Create-to-lend flows can give false inflows, and that may affect HYG and JNK. Use segments rather than single funds, because the study in Section 7 found single-fund share counts unreliable as positions.

What stops an exact replication, most serious first

1. Segment membership is proprietary.
2. Daily shares and NAV for 2014 to 2016 are needed, including closed funds.
3. The date of the AUM denominator is not stated.
4. Only one month is rebuilt from flows. The other printed months have no flow paths.
5. The share lag and the execution lag are unstated, and they may flip small changes.

6 Do the methods still work?

Next action: do not trade any signal in this document on this evidence. Run them on paper, as descriptions of positioning.

The answer is no evidence that they work. I tested the signals on May 2019 to October 2026, after the sources' samples ended. The test plan was fixed before any test ran, in `research/improvements/plan.md`. All 53 main tests failed. The smallest p-value after a Holm correction, which adjusts for running many tests, is 1.00. The smallest raw p-value is 0.030.

Table 5. Correlation of each I5 score with the next 26-week change in the 10-year yield, with Monday entry. The published column is Goldman's, through April 2019.

Signal	Published	2011 to April 2019	May 2019 to October 2026
Asset managers and others, TY and US	−43%	−27%	+4%
Asset managers and others, all six	–	−33%	−1%
Leveraged funds	−18%	−5%	−31%
Specs	−26%	−11%	−22%
Plain past 6-month yield change	−35%	−15%	+14%

A separate calculation with Tuesday entry agrees: −64% in the 2006 to 2010 fit period, −24% after it, and +5% from May 2019 for asset managers. Leveraged funds and specs look better in the later

period, but neither is significant, and doing better than the design sample is suspicious.

What failed

- **The filter creates much of its own fit.** Random flows through the same filter reach a 47% six-month correlation with yields, against 62% for real flows. The slope is −18.3 bp against −17.2 bp.
- **Pain share has no edge.** On Treasuries, leveraged funds, sell signals hit 60.2% against a 60.2% base rate. Buy signals hit 39.5% against 39.8%.
- **The removal rule does not change the answer.** That is useful, because the true rule can never be observed.
- **These additions did not help:** pain depth, pain crossed with crowding, window length, walk-forward beta, the FX filter, and combined minus futures-only as an option proxy.
- **The composite is empty.** No signal reached $|t|$ of 2 in the design sample. A version with no selection has a net Sharpe of 0.23 (t = 0.63).

> **What this result cannot say**
> A 26-week horizon gives about 14 independent observations in this period. The test would catch a true −43% correlation only 33% of the time, and 4% after the correction. So this is an absence of evidence, not proof that the signals are dead. The period was also a bear market in bonds, with rallies only 39% of the time.

Adopt these three changes

None of them adds predictive power. They remove errors at no cost.

1. **Empty-ledger guard. Done in the code.** If open contracts total 10^{-6} or less, return no reading. A full removal can leave residue near 10^{-14} contracts. In the research run, 634 of 80,956 ledgers held residue, and shares reached 2×10^{31}.
2. **Monday entry.** Measure every forward move from the close of the first trading day on or after the Monday after release. Measuring from the Tuesday report date is look-ahead. It costs at most 0.036 of correlation.
3. **Roll-free Treasury price, where no clean futures series exists.** $P_t = P_{t-1} - 100 \times DV01_{t-1}/\text{Face} \times (y_t - y_{t-1})$, using the last weekly DV01 known on the prior day. Its daily changes correlate 0.945 with real TY closes. P2 on it matches P2 on real prices (t = −1.09 against −1.04).

Watch list, on paper only

I3 on leveraged funds, read as a contrarian signal on Treasuries at 13 weeks, has a raw p-value of 0.044 and a Holm p-value of 1.00. Its slope is much weaker in the design sample, and its Sharpe goes from 1.01 in the first half to −0.52 in the second. It does not meet the adoption rule.

Ideas I have not tested

These are the changes most likely to answer the question. They are proposals, not findings.

1. **More markets.** Low power is the main limit. Pooling many futures markets adds independent observations, which waiting cannot do quickly.
2. **Real daily data for P1.** P1 has no out-of-sample test. It needs daily open interest by contract and intraday VWAP.
3. **Strike-level options for I6.** Raw OPI is the only Goldman signal never run on real data.
4. **Freeze and paper trade.** Fix the recipes now and record live signals. Each year adds evidence with no choices left to tune.

7 Credit ETFs

Next action: use the measures below to describe stress. Do not use them to predict returns.

The methods apply only in part. The ledger and the position measures compute on ETF data and describe stress. None predicts excess returns over a duration-matched Treasury hedge. Of 160 planned tests, 0 pass. The smallest Holm p-value is 0.90. The plain spread level beats every positioning signal, with t = 5.93 on LQD and 12.51 on HYG at 26 weeks. Any credit positioning study must beat that baseline. The plan, scripts and full results are in `research/credit_etf/`.

Method	Verdict	Evidence
P2	Description only	Pain depth reaches its top decile within 2 weeks of stress dates in 10 of 14 checks. The ledger covers only 8% of LQD and 13% of HYG shares.
P4	Description only	Runs with shares as longs. It makes no predictive claim.
I1	Description only	Needs real short interest. Median short interest over shares is 15% on LQD and 39% on HYG, but 0.2% on USIG.
I2	Description only	Use short interest. On share count it mostly tracks fund growth.
I3	Description only	Needs signed 26-week flow, because shares are never negative. LQD t = 2.57, HYG t = 0.01.
P1	Does not transfer	Its four cases fail against true short interest. Sign agreement is 41 to 45%.
P5	Does not transfer	Falling shares read as "Short", yet they are longs leaving.
I4	Does not transfer	Flows grow with the fund, so a fixed band fires on 72% of LQD weeks.
I5	Does not transfer	On LQD the slope falls from 103 to 18 bp without 2020, and turns negative in 2022 to 2026.
I7	Does not transfer	Continuation has the wrong sign in the cross section. Reversal flips sign in the holdout.
I8	Does not transfer	Trading the sign of momentum nets a Sharpe of -0.04 and -0.18, against 0.29 and 0.32 for buy and hold.
I6	Not tested	Option history costs money. Databento quoted $162.74 for 2018 to 2026. I did not buy it.
CFTC futures	Too short to test	Bloomberg IG and HY credit futures run unchanged in I1 to I5, but have only 26 and 24 reports.

What breaks

1. **Holders are not forced out.** Pain does not predict redemptions on HYG (t = 0.44) or JNK (t = -1.04). ETF holders are mostly not levered, so losses do not force them to sell.
2. **Shares outstanding are not a position.** Weekly share changes move against short interest, with a rank correlation of -0.38 to -0.46. So a redemption is often not a long leaving.
3. **The ledger sees a small slice.** New share creation is only 12% of volume on LQD and 9% on HYG. Read the output as the cost of recent net primary flow, not holder cost basis.
4. **Authorised participants trade the premium.** In March 2020, LQD closed 5.1% below NAV. Price and NAV disagree most when it matters most.
5. **Distributions.** A raw price overstates mean pain by 6 points on LQD, 11 on HYG and 14 on JNK. Use total return.

Recipes, for description only

Shared rules.

1. **Timing.** A row dated d is usable on the next trading day. FINRA short interest is usable from settlement plus 10 business days.
2. **Total-return NAV.** $TRN_t = TRN_{t-1}(NAV_t + \text{exdiv}_t)/NAV_{t-1}$, with $TRN_0 = 1$.
3. **Flow.** $dS_t = S_t - S_{t-1}$, from issuer shares outstanding.
4. **Order date.** A share change dated t belongs to order date $t - k$. Estimate k on past data, from 0, 1 or 2, as the lag with the highest correlation between the log share change and the premium. LQD and HYG give $k = 1$.

P2 for ETFs. Take the last 126 rows of dS ending at $T - 1$, with entry price TRN_{t-k}. Replay them with `build_ledger` and `RemovalRule.PRO_RATA`. If open total is at most 10^{-6} of gross, or of 1 if gross is smaller, return no reading. Pain share is the open shares with entry at or above TRN_{T-1}, over open total. Depth is the open-weighted mean of entry over TRN_{T-1}, minus 1, across those lots. Always publish coverage, which is open total over S_{T-1}, beside the reading.

P4 for ETFs. Call `activity_by_price` with TRN as the price, S as longs and zero shorts. Shares that leave count as longs off, not as short sales.

I1 for ETFs. $I1 = 100 \times S/(S + SI)$, with SI the FINRA short interest. Use it only where median SI/S is above 5%. That holds for LQD, HYG and JNK. It does not hold for USIG, IGSB or USHY.

I2 for ETFs. Call `sentiment_index` on $SI/(S + SI)$ with 156 weeks. FINRA's API starts on 29 December 2017. Do not run it on the level of shares.

I3 for ETFs. Compute $100(\ln S_{T-1} - \ln S_{T-1-26w})$, then call `db_scaled` with 209 weeks.

CFTC credit futures. Use codes 221606 for IG and 221605 for HY in the TFF report. Retest when I2's window fills, in about 2029.

Two open leads, neither established

1. **HY pain at 4 weeks.** HYG shows +35 bp per standard deviation (t = 2.75), and JNK agrees (t = 2.16). It holds in both sub-periods and without 2020, but it does not survive the Holm correction.
2. **LQD flows against `BAA10Y`.** 30 of 76 cells have $|t|$ above 2, where a placebo averages 5. The best Holm p-value is 0.07.

Data and its limits

- **Shares and NAV:** the iShares product-page data feed and SSGA history files. The iShares feed is undocumented, so check BlackRock's terms before production use.
- **Spreads:** FRED now keeps only 3 years of ICE BofA OAS. The study uses a spread implied from the ETF, with a duration-matched hedge. Weekly correlation with OAS is 0.92 on LQD and 0.97 on HYG. Do not read levels from it.
- **Short interest:** the FINRA API, twice monthly.
- **Sample size:** 19 independent observations at 26 weeks in the holdout, so only a very strong signal could pass there.

8 Signals and tests

8.1 S1: the natural gas Pain rule from the 2018 deck

The rule. Take open money-manager longs from the last six months. Sell NG when over 70% were initiated above the current price. Buy when under 30% were. The deck's words on the result are only that the strategy "may have some promise". It prints no performance number to match.

1. Fetch `money_mgr_long` for code `023651` from the disaggregated report, futures only.
2. Compute the P2 share in pain with $N = 126$.
3. Apply `threshold_position`: -1 above 0.70, $+1$ below 0.30.
4. Hold the last position between the two thresholds. Stay flat before the first signal.
5. Each day, use only reports already released. Trade at the next session.

> **Decisions. The source is silent, so this specification chooses.**
> The deck does not say what happens between thresholds. Holding the last signal is my choice. Run the legacy version beside it for one reconciliation: `pct_open_code` on the same data. The chart now settles the formula: its line is the survival share, on 30 rows of the combined report. The 70 and 30 thresholds were probably set on that output. Applying them to the share in pain gives a different signal. In the May 2019 to October 2026 test, the rule's hit rates equal the base rates.

8.2 How to test any signal

1. Apply the release rule in Section 2 before anything else.
2. Fix parameters on the earlier part of the sample. Goldman's splits are good models: June 2006 to 2010 for I5, and 2010 to 2015 for I6.
3. Report the base rate beside every hit rate. A 55% hit rate means little if rallies happen 55% of the time.
4. Use Newey-West errors for overlapping horizons.
5. Compare with the plain benchmark: the outright past change. Goldman shows it has a -35% correlation with the next six months.

9 Run it every day

Next action: copy this schedule into your scheduler, starting with the weekly job.

When	Job	Runs
Daily, after final settlement and open interest	Load `market_daily`, run the data checks	P1, P5, I6, and the daily price update of P2
Friday from 15:35 New York, poll every 5 minutes until a new report date appears	Load `cftc_weekly` with `first_seen`	P2, I1 to I5, I7
Thursday close	Load factor inputs	I10, for the Friday morning trade
Monday before the open	Publish weekly signals	S1 and I7 orders
Quarterly	Rerun `reconcile_live.py` and compare with Table 6	Catches feed and schema changes

Commands

```
python -m posmon.cli fetch --report disaggregated --code 023651 --start 2006-01-01 --out store/
    dis_023651.csv
python -m posmon.cli p1 --daily store/ty_daily.csv --multiplier 1000 --out out/p1_ty.csv
```

```
python -m posmon.cli p2 --weekly store/dis_023651.csv --field money_mgr_long --daily store/ng_daily.
    csv --window 126 --out out/p2_ng.csv
python -m posmon.cli si --report store/leg_043602.csv --long noncomm_long --short noncomm_short --out
    out/si_ty.csv
```

A daily file needs the columns `date`, `close`, `high`, `low`, `volume` and `open_interest`. An optional `vwap` column overrides the typical-price fallback.

What to store for every output row

Store the as-of date, contract, method ID and side. Store the window, removal rule, VWAP source and report basis. Store the latest report date used, its `first_seen` time and the code version. Without these, you cannot explain an old number later. This whole exercise had to solve that problem.

Alerts

1. A ledger invariant fails. Stop that contract.
2. A CFTC identity fails, or a week is missing. Stop the weekly job.
3. No new report by Monday 09:00 New York. Hold the last signal and tell a person.
4. The unmatched amount exceeds the open amount. The reading is mostly blind to older positions, so flag it.
5. A CFTC column goes missing. `fetch_cftc` raises an error, so the CFTC has probably renamed it. Update `COLUMNS`.

The fourth alert will fire often on large contracts. On TY at 22 January 2015, long unmatched was 49,683 against 90,152 open. So these monitors describe recent flow, not the whole market.

10 Accept the build

Next action: tick each row. The build is accepted when your numbers equal the Rebuilt column of Table 6 and `pytest` shows 72 passed.

Blind test of this document

An independent agent built the methods from the text alone, with no code and no answer boxes, in about 10 minutes. It matched 121 of 131 held-back values at first attempt. All 10 misses were in I10. Nine came from unclear wording on where the cap applies, and one from unclear momentum wording. With the cap read as intended, those values agree. It also listed about 30 places where the text forced a guess. I fixed those that had a real effect, including the release rule missing from P2's code. I did not run a second blind test on the corrected text.

What this means for you. P3, P4, P5, I1, I2, I3 and I5 reproduce their sources. P1, P2 and I11 are complete, but nothing exists to match them against. I4, I6, I7, I8, I10 and I13 reproduce in part. I9 cannot be reproduced. None of this shows that a signal makes money, so read Section 6 first.

11 Where the sources disagree

Table 6. Live reconciliation, from `data/reconcile.json`. Rebuilt on 1 November 2026. Small changes may appear if the CFTC revises history.

Check	Source	Rebuilt	Result
Workbook positions against the CFTC API, 350 cells	same	0 differ	Match
Workbook open interest against the CFTC, 25 dates	same	2 differ	Close
EUR sentiment index, 24 Jul 2012	18.9	18.91	Match
2-year sentiment index, 12 Jul 2016	64	63.5	Match
10-year sentiment index, 12 Jul 2016	3-year high	100.0	Match
2-year weeks at zero, around end-May 2016	3	2	Gap
TY net non-commercial, 9 May 2017	229K	229,119	Match
Eurodollar net as % of open interest, 9 May 2017	−24%	−23.6%	Match
Deutsche Bank scaled chart, mean error over 22 values	0	0.04	Match
Goldman filter slope, bp per $10mm	17–18	−17.2	Match
Goldman filter against its own charts, leveraged funds and asset managers	100%	99.98%, 99.91%	Match
Goldman filter, next-6m correlation, asset managers	−43%	−43%	Match
Goldman filter, last score, 30 Apr 2019	−0.67	−0.66	Match
One-standard-deviation rule, base rate	53%	53.7%	Close
One-standard-deviation rule, buy and sell hit rates	63%, 48%	58.1%, 45.6%	Gap
Natural gas ladder at every ratio, read from the deck	about 69,000	69,322	Match
Morgan Stanley BMI combined signals, four markets	printed	all four equal	Match
Workbook P5, all formula columns	cached	gap under 10^{-13}	Match

Question	What the sources say	Decision
How do reductions leave the book?	Morgan Stanley: proportionately. 2015 emails: uniformly. Deck: FIFO. 2018 text: lower prices fastest. Code: higher prices lose most.	Pro rata.
Does “uniformly” mean equal amounts or equal shares?	The emails say all open positions sell uniformly. Morgan Stanley says proportionately.	Equal shares.
Entry price?	Morgan Stanley: daily VWAP. PortfolioStrat: close of the day. Workbook: typical price.	Keep each per method.
Which position data?	Morgan Stanley: price and open interest. PortfolioStrat: CFTC fields.	Separate methods, P1 and P2.
How long is the window?	Morgan Stanley 30 days. Code and screen 30 rows. Description 90 rows. Deck six months. Workbook 30 and 29 rows.	Per method. P2 signals use 126.
What does the Pain percentage mean?	Deck: share of open positions above the price. Code: survival share above the price.	Deck’s line: survival share, 30 rows, combined report. Trade the share in pain.
Futures only, or with options?	Old loader and Goldman I5: combined. Workbook, Deutsche Bank and Goldman I2: futures only.	Per method, as Section 2 states.
OPI signal direction?	Goldman page 4 against its summary and Exhibit 3.	Upper band means sell.

Still open

1. GLOBAL_20200614_0000.pdf is still undownloaded. It may hold more on this topic. You can save it from Gmail by hand.
2. The Morgan Stanley contract charts are not in the scan. They would let P1 be verified. P1 also has no out-of-sample test, because it needs daily open interest and VWAP.
3. The saved NG workspace and painmonitor.ipynb were not found. The chart settles the formula, but not the window of the backtest panel.
4. Goldman's asset manager series loads only on TY and US. The report does not say why.
5. Strike-level option history is needed to run I6 on real markets.
6. I11's sampler has not run. It needs CmdStan and 40 model forms on 136 series.

A Reference code

These are the tested files. Copy them from spec/posmon, not from this PDF. They appear here so the rules can be read in one place.

A.1 ledger.py

```
"""Lot ledger: the engine shared by every Pain method.

A ledger holds the positions added inside one window. Each addition becomes a lot
with an entry price. Each reduction removes contracts from earlier lots under one
removal rule. The ledger is rebuilt from empty for every as-of date.
"""
from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum
from typing import Iterable, Sequence


class RemovalRule(str, Enum):
    """How a reduction is taken out of the open lots."""

    PRO_RATA = "pro_rata"  # Morgan Stanley 2014 and the June 2015 emails
    FIFO = "fifo"  # the screenshot on page 16 of the 2018 deck
    LOW_PRICE_FIRST = "low_price_first"  # the May 2018 function description


@dataclass
class Lot:
    date: object
    entry: float
    gross: float
    open: float


@dataclass
class Ledger:
    lots: list = field(default_factory=list)
    requested: float = 0.0  # total reduction asked for
    removed: float = 0.0  # total reduction matched to lots
    unmatched: float = 0.0  # reduction with no lot to take it from

    @property
    def gross_total(self) -> float:
        return sum(lot.gross for lot in self.lots)

    @property
    def open_total(self) -> float:
        return sum(lot.open for lot in self.lots)
```

```
    def open_above(self, level: float, inclusive: bool = True) -> float:
        """Open contracts with an entry price above the level."""
        if inclusive:
            return sum(lot.open for lot in self.lots if lot.entry >= level)
        return sum(lot.open for lot in self.lots if lot.entry > level)

    def open_below(self, level: float, inclusive: bool = True) -> float:
        """Open contracts with an entry price below the level."""
        if inclusive:
            return sum(lot.open for lot in self.lots if lot.entry <= level)
        return sum(lot.open for lot in self.lots if lot.entry < level)

    def check(self, tolerance: float = 1e-6) -> None:
        """Raise if a ledger invariant fails. Call this in production."""
        for lot in self.lots:
            if lot.open < -tolerance or lot.open > lot.gross + tolerance:
                raise AssertionError(f"lot outside 0..gross: {lot}")
        balance = self.gross_total - self.removed - self.open_total
        if abs(balance) > tolerance * max(1.0, self.gross_total):
            raise AssertionError(f"gross - removed != open, gap {balance}")
        gap = self.requested - self.removed - self.unmatched
        if abs(gap) > tolerance * max(1.0, self.requested):
            raise AssertionError(f"requested != removed + unmatched, gap {gap}")


def _remove_pro_rata(lots: list, amount: float) -> None:
    total = sum(lot.open for lot in lots)
    # The denominator stays fixed for the whole loop. The legacy code let it shrink.
    for lot in lots:
        lot.open -= amount * lot.open / total


def _remove_in_order(lots: Iterable, amount: float) -> None:
    left = amount
    for lot in lots:
        taken = min(lot.open, left)
        lot.open -= taken
        left -= taken
        if left <= 0:
            break


def build_ledger(
    changes: Sequence[float],
    entries: Sequence[float],
    rule: RemovalRule = RemovalRule.PRO_RATA,
    dates: Sequence | None = None,
) -> Ledger:
    """Replay one window of signed changes, oldest first."""
    if len(changes) != len(entries):
        raise ValueError("changes and entries must have the same length")
    dates = list(dates) if dates is not None else list(range(len(changes)))
    ledger = Ledger()
    for stamp, change, entry in zip(dates, changes, entries):
        if change != change or change == 0:  # NaN or no change
            continue
        if change > 0:
            ledger.lots.append(Lot(stamp, float(entry), float(change), float(change)))
            continue
        asked = -float(change)
        ledger.requested += asked
        available = ledger.open_total
        taken = min(asked, available)
        ledger.removed += taken
        ledger.unmatched += asked - taken
        if taken <= 0:
            continue
        live = [lot for lot in ledger.lots if lot.open > 0]
        if rule is RemovalRule.PRO_RATA:
            _remove_pro_rata(live, taken)
        elif rule is RemovalRule.FIFO:
            _remove_in_order(live, taken)
        elif rule is RemovalRule.LOW_PRICE_FIRST:
            _remove_in_order(sorted(live, key=lambda lot: lot.entry), taken)
```

```
        else:
            raise ValueError(f"unknown rule {rule}")
    ledger.check()
    return ledger


def bucket_lots(ledger: Ledger, window_low: float, window_high: float, buckets: int = 10) -> list:
    """Group lots into equal price buckets for a chart. Do not use buckets for the signal.

    Each row is (lower edge, gross, removed, open). The top price falls in the last bucket.
    """
    if window_high <= window_low:
        raise ValueError("flat window: the buckets have zero width")
    width = (window_high - window_low) / buckets
    rows = [[window_low + i * width, 0.0, 0.0, 0.0] for i in range(buckets)]
    for lot in ledger.lots:
        index = min(int((lot.entry - window_low) / width), buckets - 1)
        rows[index][1] += lot.gross
        rows[index][2] += lot.gross - lot.open
        rows[index][3] += lot.open
    return rows
```

A.2 pain.py

```
"""The Pain methods: P1 Morgan Stanley, P2 CFTC corrected, P3 legacy port, P4 activity, P5 workbook."""
from __future__ import annotations

import math

import numpy as np
import pandas as pd

from .data import publish_time
from .ledger import Ledger, RemovalRule, build_ledger

# ---------------------------------------------------------------------------
# Prices
# ---------------------------------------------------------------------------


def typical_price(high: pd.Series, low: pd.Series, close: pd.Series) -> pd.Series:
    """Average of high, low and close. Missing inputs are skipped, as Excel AVERAGE does."""
    return pd.concat([high, low, close], axis=1).mean(axis=1, skipna=True)


def vwap_from_bars(bars: pd.DataFrame) -> pd.Series:
    """Daily VWAP from intraday bars with columns date, price and volume."""
    value = (bars["price"] * bars["volume"]).groupby(bars["date"]).sum()
    return value / bars["volume"].groupby(bars["date"]).sum()


def back_adjust(front: pd.Series, gaps: pd.Series) -> pd.Series:
    """Difference back-adjustment.

    gaps holds (new contract - old contract) at each roll close, indexed by the roll date.
    Prices strictly before a roll date move by that gap. The latest price never moves.
    """
    adjusted = front.astype(float).copy()
    for roll_date, gap in gaps.items():
        adjusted.loc[adjusted.index < roll_date] += gap
    return adjusted


# ---------------------------------------------------------------------------
# P1: Morgan Stanley Pain Trade Monitor
# ---------------------------------------------------------------------------


def classify_changes(close: pd.Series, open_interest: pd.Series) -> pd.DataFrame:
    """Turn price and open-interest changes into signed long and short changes."""
    price_change = close.diff()
    oi_change = open_interest.diff()
    up, down = price_change > 0, price_change < 0
```

```
    rise, fall = oi_change > 0, oi_change < 0
    frame = pd.DataFrame({"price_change": price_change, "oi_change": oi_change}, index=close.index)
    frame["long_change"] = 0.0
    frame["short_change"] = 0.0
    frame.loc[up & rise, "long_change"] = oi_change[up & rise]  # new longs
    frame.loc[down & fall, "long_change"] = oi_change[down & fall]  # longs taken off
    frame.loc[down & rise, "short_change"] = oi_change[down & rise]  # new shorts
    frame.loc[up & fall, "short_change"] = oi_change[up & fall]  # shorts covered
    frame["case"] = "none"
    frame.loc[up & rise, "case"] = "new_long"
    frame.loc[down & fall, "case"] = "long_off"
    frame.loc[down & rise, "case"] = "new_short"
    frame.loc[up & fall, "case"] = "short_cover"
    frame.loc[price_change.isna() | oi_change.isna(), "case"] = "first_row"
    return frame


def ms_pain_monitor(
    close: pd.Series,
    vwap: pd.Series,
    open_interest: pd.Series,
    multiplier: float,
    fx_to_usd: float = 1.0,
    window: int = 30,
    rule: RemovalRule = RemovalRule.PRO_RATA,
    as_of=None,
) -> dict:
    """Compute one as-of reading of the Morgan Stanley monitor.

    The window holds `window` daily changes, so it needs window + 1 rows of levels.
    A long lot is offside when its entry is above the as-of close.
    A short lot is offside when its entry is below the as-of close.
    """
    as_of = close.index[-1] if as_of is None else as_of
    classified = classify_changes(close.loc[:as_of], open_interest.loc[:as_of])
    if len(classified) < window + 1:
        raise ValueError(f"need {window + 1} rows, got {len(classified)}")
    tail = classified.iloc[-window:]
    entries = vwap.reindex(tail.index)
    long_book = build_ledger(tail["long_change"].values, entries.values, rule, tail.index)
    short_book = build_ledger(tail["short_change"].values, entries.values, rule, tail.index)
    level = float(close.loc[as_of])
    oi_now = float(open_interest.loc[as_of])

    def notional(lots) -> float:
        return sum(lot.open * lot.entry * multiplier * fx_to_usd for lot in lots)

    long_off = [lot for lot in long_book.lots if lot.entry > level]
    short_off = [lot for lot in short_book.lots if lot.entry < level]
    long_off_qty = sum(lot.open for lot in long_off)
    short_off_qty = sum(lot.open for lot in short_off)
    return {
        "as_of": as_of,
        "close": level,
        "open_interest": oi_now,
        "long_open": long_book.open_total,
        "short_open": short_book.open_total,
        "long_offside": long_off_qty,
        "short_offside": short_off_qty,
        "long_offside_pct_new": _ratio(long_off_qty, long_book.open_total),
        "short_offside_pct_new": _ratio(short_off_qty, short_book.open_total),
        "long_offside_pct_oi": long_off_qty / oi_now,
        "short_offside_pct_oi": short_off_qty / oi_now,
        "long_offside_usd": notional(long_off),
        "short_offside_usd": notional(short_off),
        "long_unmatched": long_book.unmatched,
        "short_unmatched": short_book.unmatched,
        "long_book": long_book,
        "short_book": short_book,
    }


def rank_monitor(readings: dict, side: str = "long") -> pd.DataFrame:
    """Rank contracts by the offside share of total open interest. Larger comes first."""
```

```
    column = f"{side}_offside_pct_oi"
    rows = [{"contract": name, column: value[column], f"{side}_offside_usd": value[f"{side}_offside_usd"]}
            for name, value in readings.items()]
    return pd.DataFrame(rows).sort_values(column, ascending=False).reset_index(drop=True)


def _ratio(top: float, bottom: float, floor: float = 1e-6) -> float:
    """A full removal can leave floating-point residue near 1e-14 contracts. Treat that as an empty ledger."""
    return top / bottom if bottom > floor else float("nan")


# ---------------------------------------------------------------------------
# P2: PortfolioStrat method, corrected
# ---------------------------------------------------------------------------


def harmonise_weekly(position: pd.Series, trading_days: pd.DatetimeIndex) -> pd.Series:
    """Spread a weekly CFTC field onto trading days. A change lands on its report date.

    A report date that is not a trading day moves to the next trading day.
    """
    union = trading_days.union(position.index)
    return position.reindex(union).ffill().reindex(trading_days)


def cftc_pain(
    position: pd.Series,
    close: pd.Series,
    window: int = 126,
    rule: RemovalRule = RemovalRule.PRO_RATA,
    side: str = "long",
    respect_release: bool = True,
) -> pd.DataFrame:
    """Roll the ledger over one CFTC field and return one row per as-of date.

    With respect_release, an as-of date sees only reports released on or before it.
    Set it to False only to reconcile with old output, which used reports from their report date.

    share_in_pain: open contracts entered at or beyond the close, as a share of all open contracts.
    For a long field, beyond means at or above. For a short field, it means at or below.
    pct_open_code: the same price test, but open divided by gross. This is what PCT_OPEN returns.
    """
    daily = harmonise_weekly(position, close.index)
    frame = pd.DataFrame({"close": close, "change": daily.diff()}).dropna()
    released = pd.Series(pd.NaT, index=frame.index, dtype="datetime64[ns]")
    for report_date in position.index:
        spot = frame.index.searchsorted(report_date)
        if spot < len(frame.index):
            released.iloc[spot] = publish_time(report_date).tz_localize(None).normalize()
    rows = []
    for end in range(window, len(frame) + 1):
        part = frame.iloc[end - window:end]
        changes = part["change"]
        if respect_release:
            changes = changes.where(~(released.iloc[end - window:end] > part.index[-1]), 0.0)
        book = build_ledger(changes.values, part["close"].values, rule, part.index)
        level = float(part["close"].iloc[-1])
        rows.append({"date": part.index[-1], "close": level, "open": book.open_total,
                     "gross": book.gross_total, "unmatched": book.unmatched,
                     "share_in_pain": _share_in_pain(book, level, side),
                     "pct_open_code": _survival_share(book, level, side)})
    return pd.DataFrame(rows).set_index("date")


def _share_in_pain(book: Ledger, level: float, side: str) -> float:
    held = book.open_above(level) if side == "long" else book.open_below(level)
    return _ratio(held, book.open_total)


def _survival_share(book: Ledger, level: float, side: str) -> float:
    if side == "long":
        lots = [lot for lot in book.lots if lot.entry >= level]
    else:
        lots = [lot for lot in book.lots if lot.entry <= level]
```

```
    gross = sum(lot.gross for lot in lots)
    return sum(lot.open for lot in lots) / gross if gross else 0.0


def threshold_position(signal: pd.Series, sell_above: float = 0.70, buy_below: float = 0.30) -> pd.Series:
    """Deck rule for NG. Sell above 70 percent. Buy below 30 percent. Hold the last signal between."""
    position = pd.Series(np.nan, index=signal.index)
    position[signal > sell_above] = -1.0
    position[signal < buy_below] = 1.0
    return position.ffill().fillna(0.0)


# ---------------------------------------------------------------------------
# P3: exact port of the legacy function, defects included
# ---------------------------------------------------------------------------


def legacy_position_per_strike(position_changes, prices) -> dict:
    """Port of _calculate_position_per_strike (functions.py lines 1015 to 1050). Do not fix it."""
    net_position = 0
    bucket_count = 10
    min_price = min(prices)
    max_price = max(prices)
    bucket_width = (max_price - min_price) / float(bucket_count)
    positions = [{"price": min_price + float(i) * bucket_width, "gross": 0, "liquidated": 0}
                 for i in range(bucket_count)]
    for px, ch in zip(prices, position_changes):
        which_bucket = int(math.floor((px - min_price) / bucket_width))
        if px == max_price:
            which_bucket -= 1
        if ch > 0:
            net_position += ch
            positions[which_bucket]["gross"] += ch
        elif ch < 0 < net_position:
            for p in positions:
                to_liquidate = 0
                if p["gross"] > p["liquidated"] and net_position > 0:
                    potential = p["gross"] - p["liquidated"]
                    to_liquidate = min(potential / net_position * ch * -1, potential)
                    p["liquidated"] += to_liquidate
                net_position = max(0, net_position - to_liquidate)
    positions.sort(key=lambda x: x["price"])
    return {"prices": [x["price"] for x in positions],
            "open": [x["gross"] - x["liquidated"] for x in positions],
            "gross": [x["gross"] for x in positions],
            "closed": [x["liquidated"] for x in positions]}


def legacy_cftc_open_position(field: str, prices: pd.Series, positions: pd.Series, level,
                              condition: str = "==", window: int = 30) -> pd.Series:
    """Port of f_cftc_open_position (functions.py lines 961 to 1011). Do not fix it."""
    field = str(field).strip().upper()
    if field not in ("OPEN", "GROSS", "CLOSED", "PCT_OPEN"):
        raise ValueError(field)
    if condition not in ("==", ">=", "<="):
        raise ValueError(condition)
    levels = level if isinstance(level, pd.Series) else pd.Series(float(level), index=prices.index)
    frame = pd.DataFrame({"prices": prices, "position_changes": positions - positions.shift(1),
                          "levels": levels}).dropna()

    def one(part: pd.DataFrame) -> float:
        book = legacy_position_per_strike(part["position_changes"].values, part["prices"].values)
        last = part["levels"].iloc[-1]
        zipped_open = list(zip(book["prices"], book["open"]))
        zipped_gross = list(zip(book["prices"], book["gross"]))
        if condition == "==":
            if field == "PCT_OPEN":
                remaining = np.interp(last, book["prices"], book["open"])
                gross = np.interp(last, book["prices"], book["gross"])
                return remaining / gross if gross else 0.0
            return float(np.interp(last, book["prices"], book[field.lower()]))
        if condition == ">=":
            if field == "PCT_OPEN":
                remaining = sum(z[1] if z[0] >= last else 0.0 for z in zipped_open)
```

```
                gross = sum(z[1] if z[0] >= last else 0.0 for z in zipped_gross)
                return float(remaining) / float(gross) if gross else 0.0
            return float(sum(x if x >= last else 0.0 for x in book[field.lower()]))  # compares size to price
        if field == "PCT_OPEN":
            remaining = sum(z[1] if z[0] <= last else 0.0 for z in zipped_open)
            gross = sum(z[1] if z[0] <= last else 0.0 for z in zipped_gross)
            return float(remaining) / float(gross) if gross else 0.0
        return float(sum(x if x <= last else 0.0 for x in book[field.lower()]))  # compares size to price

    results = [one(frame.iloc[i:i + window]) for i in range(len(frame) - window + 1)]
    return pd.Series(results, index=frame.index[window - 1:])


def legacy_ladder(position_changes, prices, current_price, ratios) -> pd.DataFrame:
    """Rebuild the deck ladder chart with the legacy OPEN >= path and with a correct price test."""
    book = legacy_position_per_strike(position_changes, prices)
    rows = []
    for ratio in ratios:
        level = ratio * current_price
        wrong = sum(x if x >= level else 0.0 for x in book["open"])
        right = sum(o for p, o in zip(book["prices"], book["open"]) if p >= level)
        rows.append({"ratio": ratio, "level": level, "legacy_open": wrong, "correct_open": right})
    return pd.DataFrame(rows)


# ---------------------------------------------------------------------------
# P4: activity by trade price (the email exchange of June 2015)
# ---------------------------------------------------------------------------


def activity_by_price(price, longs, shorts) -> pd.DataFrame:
    """Place every change at the price of the day it happened. Entry and exit are not matched."""
    frame = pd.DataFrame({"price": price, "long_chg": pd.Series(longs).diff(),
                          "short_chg": pd.Series(shorts).diff()}).dropna()
    out = pd.DataFrame({
        "price": frame["price"],
        "longs_added": frame["long_chg"].clip(lower=0),
        "longs_off": (-frame["long_chg"]).clip(lower=0),
        "shorts_added": frame["short_chg"].clip(lower=0),
        "shorts_off": (-frame["short_chg"]).clip(lower=0),
    }).groupby("price").sum()
    out["buys"] = out["longs_added"] + out["shorts_off"]
    out["sells"] = out["shorts_added"] + out["longs_off"]
    out["longs_net"] = out["longs_added"] - out["longs_off"]
    out["shorts_net"] = out["shorts_added"] - out["shorts_off"]
    return out


# ---------------------------------------------------------------------------
# P5: the 2018 workbook sheet
# ---------------------------------------------------------------------------


def workbook_over_under(daily: pd.DataFrame, vwap_rows: int = 30, net_rows: int = 29) -> pd.DataFrame:
    """Rebuild columns G to P of the "Pain Monitor" sheet.

    The sheet calls M a VWAP. It is a rolling volume-weighted mean of typical price.
    A missing volume counts as zero, as SUMPRODUCT and SUM treat text.
    """
    out = pd.DataFrame(index=daily.index)
    out["typical"] = typical_price(daily["high"], daily["low"], daily["close"])
    out["price_change"] = out["typical"].diff()
    out["oi_change"] = daily["open_interest"].diff()
    volume = daily["volume"].fillna(0.0)
    out["oi_pct_volume"] = out["oi_change"] / daily["volume"]
    value = (volume * out["typical"]).rolling(vwap_rows, min_periods=1).sum()
    out["vwap"] = value / volume.rolling(vwap_rows, min_periods=1).sum()
    out.loc[out.index[0], "vwap"] = np.nan  # the sheet starts this column on its second row
    out["net_new"] = out["oi_change"].rolling(net_rows, min_periods=1).sum()
    out["side"] = np.where(out["net_new"] > 0, "Long", "Short")
    out.loc[out.index[0], "side"] = None
    out["over_under"] = out["typical"] - out["vwap"]
    out["underwater"] = ((out["side"] == "Long") & (out["over_under"] < 0)) | (
        (out["side"] == "Short") & (out["over_under"] > 0))
```

```
    return out
```

A.3 indicators.py

```
"""Positioning indicators that do not estimate entry prices."""
from __future__ import annotations

import math

import numpy as np
import pandas as pd
from scipy.stats import norm

# ---------------------------------------------------------------------------
# I1: net position as a share of open interest
# ---------------------------------------------------------------------------


def net_pct_oi(long_side: pd.Series, short_side: pd.Series, open_interest: pd.Series) -> pd.Series:
    return 100.0 * (long_side - short_side) / open_interest


def rolling_stats(series: pd.Series, weeks: int = 52) -> pd.DataFrame:
    """The 12-month high, low and median used in the Deutsche Bank sales note."""
    roll = series.rolling(weeks, min_periods=weeks)
    return pd.DataFrame({"high": roll.max(), "low": roll.min(), "median": roll.median()})


# ---------------------------------------------------------------------------
# I2: Goldman Sachs sentiment index
# ---------------------------------------------------------------------------


def sentiment_index(net: pd.Series, weeks: int = 156) -> pd.Series:
    """Scale net position between its rolling low and high. The window includes today."""
    low = net.rolling(weeks, min_periods=weeks).min()
    high = net.rolling(weeks, min_periods=weeks).max()
    return 100.0 * (net - low) / (high - low)


def extreme_flags(si: pd.Series, high: float = 90.0, low: float = 10.0) -> pd.DataFrame:
    """Dummies D90 and D10, and the counter of weeks spent in the current extreme."""
    d90 = (si >= high).astype(float).where(si.notna())
    d10 = (si <= low).astype(float).where(si.notna())
    state = d90.fillna(0) - d10.fillna(0)
    run_id = (state != state.shift()).cumsum()
    counter = state.abs().groupby(run_id).cumsum()
    return pd.DataFrame({"d90": d90, "d10": d10, "counter": counter})


# ---------------------------------------------------------------------------
# I3: Deutsche Bank scaled positioning
# ---------------------------------------------------------------------------


def db_scaled(net: pd.Series, weeks: int = 209) -> pd.Series:
    """Map net contracts to -10..+10 over four years, keeping zero at zero.

    +10 is the largest net long in the window. -10 is the largest net short.
    A window with no net long has no positive scale. The same holds for shorts.
    """
    largest_long = net.rolling(weeks, min_periods=weeks).max().clip(lower=0)
    largest_short = net.rolling(weeks, min_periods=weeks).min().clip(upper=0).abs()
    up = 10.0 * net / largest_long.where(largest_long > 0)
    down = 10.0 * net / largest_short.where(largest_short > 0)
    out = pd.Series(np.where(net > 0, up, np.where(net < 0, down, 0.0)), index=net.index)
    return out.where(largest_long.notna())


# ---------------------------------------------------------------------------
# I4: one-standard-deviation rule on duration-weighted net position
# ---------------------------------------------------------------------------
```

```
def dv01_weighted_net(nets: dict, dv01: dict) -> pd.Series:
    """Sum of net contracts times dollars per basis point, in $mm per basis point."""
    frame = pd.DataFrame({key: nets[key] * dv01[key] / 1e6 for key in nets})
    return frame.sum(axis=1, min_count=1)


def one_sd_rule(position: pd.Series, multiple: float = 1.0) -> pd.Series:
    """+1 is a buy signal after a bearish extreme. -1 is a sell signal after a bullish extreme."""
    score = (position - position.mean()) / position.std()
    return pd.Series(np.where(score <= -multiple, 1.0, np.where(score >= multiple, -1.0, 0.0)),
                     index=position.index)


# ---------------------------------------------------------------------------
# I5: Goldman Sachs filtered position and residual
# ---------------------------------------------------------------------------


def filtered_change(nets: dict, yields_bp: pd.DataFrame, dv01) -> pd.Series:
    """Keep a tenor's change only when it moves with the market, then sum in $mm/bp.

    With the market means net selling as yields rise, or net buying as yields fall.
    dv01[tenor] is a fixed number or a weekly series of dollars per contract.
    """
    total = None
    for tenor, net in nets.items():
        change = net.diff()
        move = yields_bp[tenor].reindex(net.index, method="ffill").diff()
        weight = dv01[tenor]
        if isinstance(weight, pd.Series):
            weight = weight.reindex(net.index, method="ffill")
        kept = (change * weight / 1e6).where(np.sign(change) == -np.sign(move), 0.0).fillna(0.0)
        total = kept if total is None else total.add(kept, fill_value=0.0)
    return total


def filtered_residual(yield_10y_bp: pd.Series, filtered: pd.Series, fit_start="2006-06-13",
                      fit_end="2010-12-31", weeks: int = 26, trim: float = 0.25) -> pd.DataFrame:
    """Fit on the in-sample window, sum residuals over six months, and score them.

    A positive score means yields rose by more than positioning implies. That is a rally bias.
    The score divides the six-month sum by the in-sample weekly residual spread times sqrt(weeks).
    It subtracts no mean, and it uses no data after fit_end, so it has no look-ahead after that date.
    """
    level = yield_10y_bp.reindex(filtered.index, method="ffill")
    move = level.diff()
    mask = (filtered.index >= pd.Timestamp(fit_start)) & (filtered.index <= pd.Timestamp(fit_end))
    mask &= move.notna().values
    slope, intercept = np.polyfit(filtered[mask], move[mask], 1)
    residual = move - (intercept + slope * filtered)
    summed = residual.rolling(weeks).sum()
    scale = residual[mask].std() * np.sqrt(weeks)
    score = summed / scale
    signal = pd.Series(np.where(score > trim, 1.0, np.where(score < -trim, -1.0, 0.0)), index=score.index)
    return pd.DataFrame({"slope": slope, "intercept": intercept, "residual": residual, "scale": scale,
                         "residual_6m": summed, "score": score, "signal": signal.where(score.notna())})


def forward_stats(level_bp: pd.Series, signal: pd.Series, weeks: int = 26) -> dict:
    """Hit rates against the change in yield over the next six months."""
    ahead = level_bp.shift(-weeks) - level_bp
    usable = ahead.notna() & signal.notna()
    long_case = usable & (signal > 0)
    short_case = usable & (signal < 0)
    return {"total": int(usable.sum()), "uncond_rally": float((ahead[usable] < 0).mean()),
            "uncond_avg": float(ahead[usable].mean()), "pos_n": int(long_case.sum()),
            "pos_hit": float((ahead[long_case] < 0).mean()), "pos_avg": float(ahead[long_case].mean()),
            "neg_n": int(short_case.sum()), "neg_hit": float((ahead[short_case] > 0).mean()),
            "neg_avg": float(ahead[short_case].mean())}


# ---------------------------------------------------------------------------
# I6: Goldman Sachs options-implied position indicator
```

```
# ---------------------------------------------------------------------------


def black76_abs_delta(forward, strike, vol, years, rate, is_call) -> float:
    """Absolute Black-76 delta. The formula for OAI needs positive deltas."""
    d1 = (math.log(forward / strike) + 0.5 * vol * vol * years) / (vol * math.sqrt(years))
    discount = math.exp(-rate * years)
    return discount * (norm.cdf(d1) if is_call else norm.cdf(-d1))


def select_options(chain: pd.DataFrame, half_width: float = 10.0, expiries: int = 3) -> pd.DataFrame:
    """Keep the front three expiries and strikes within 10 points of the forward.

    Columns: date, expiry, strike, is_call, forward, abs_delta, oi_change, volume.
    """
    chain = chain[(chain["strike"] - chain["forward"]).abs() <= half_width].copy()
    ranks = chain.groupby("date")["expiry"].rank(method="dense")
    return chain[ranks <= expiries]


def option_components(chain: pd.DataFrame) -> pd.DataFrame:
    """Compute the four daily delta-weighted sums that feed OAI and OPI."""
    weighted = chain.assign(w_oi=chain["abs_delta"] * chain["oi_change"], w_v=chain["abs_delta"] * chain["volume"])
    calls = weighted[weighted["is_call"]].groupby("date")[["w_oi", "w_v"]].sum()
    puts = weighted[~weighted["is_call"]].groupby("date")[["w_oi", "w_v"]].sum()
    out = pd.DataFrame({"call_oi": calls["w_oi"], "put_oi": puts["w_oi"], "call_v": calls["w_v"], "put_v": puts["
     w_v"]})
    return out.sort_index().fillna(0.0)


def oai(parts: pd.DataFrame) -> pd.Series:
    return (parts["call_oi"] - parts["put_oi"]) / (parts["call_v"] + parts["put_v"])


def opi(parts: pd.DataFrame, half_life: float = 25.0) -> pd.Series:
    """Smooth each part first. The ratio of averages keeps low-volume days small."""
    ema = parts.ewm(halflife=half_life, adjust=False).mean()
    return (ema["call_oi"] - ema["put_oi"]) / (ema["call_v"] + ema["put_v"])


def opi_signal(indicator: pd.Series, band_half_life: float = 63.0, multiple: float = 0.85,
               min_periods: int = 20) -> pd.DataFrame:
    """Sell above the upper band and buy below the lower band. Bands use yesterday's data.

    The centre is an exponential mean. The half-width is k times the expanding standard deviation.
    Goldman's charts fix this: width correlates 0.9994 and 0.9987, with implied k of 0.846 and 1.242.
    """
    mean = indicator.ewm(halflife=band_half_life, adjust=False).mean().shift(1)
    std = indicator.expanding(min_periods).std().shift(1)
    upper, lower = mean + multiple * std, mean - multiple * std
    signal = pd.Series(np.where(indicator > upper, -1.0, np.where(indicator < lower, 1.0, 0.0)), index=indicator.
     index)
    return pd.DataFrame({"opi": indicator, "upper": upper, "lower": lower, "signal": signal.where(upper.notna())})


# ---------------------------------------------------------------------------
# I7: Deutsche Bank FX Cookbook signals
# ---------------------------------------------------------------------------


def cftc_continuation(long_side: pd.Series, short_side: pd.Series, weeks: int = 4) -> pd.Series:
    """Four-week sums of longs and shorts, as a share of their total."""
    longs = long_side.rolling(weeks, min_periods=weeks).sum()
    shorts = short_side.rolling(weeks, min_periods=weeks).sum()
    return (longs - shorts) / (longs + shorts)


def cftc_reversal(net: pd.Series, daily_returns: pd.Series, lookbacks=(4, 8, 13), vol_days: int = 126) -> pd.Series
    :
    """Average z-score over one, two and three months, over six-month volatility, with the sign flipped."""
    scores = [(net - net.rolling(n, min_periods=n).mean()) / net.rolling(n, min_periods=n).std() for n in lookbacks
     ]
    average = pd.concat(scores, axis=1).mean(axis=1, skipna=False)
    volatility = daily_returns.rolling(vol_days, min_periods=vol_days).std()
```

```
    return -average / volatility.reindex(average.index, method="ffill")


def coffee_signal(call_notional: pd.Series, put_notional: pd.Series, sum_days: int = 20, vol_days: int = 252) -> pd
    .Series:
    """Four-week call minus put notional, over the one-year volatility of that measure."""
    imbalance = (call_notional - put_notional).rolling(sum_days, min_periods=sum_days).sum()
    return imbalance / imbalance.rolling(vol_days, min_periods=vol_days).std()


def time_series_weights(signals: pd.Series, vols: pd.Series) -> pd.Series:
    """Sign of the signal times an inverse-volatility share."""
    inverse = 1.0 / vols
    return np.sign(signals) * inverse / inverse.sum()


def cross_section_weights(signals: pd.Series) -> pd.Series:
    """Long the top half and short the bottom half. Absolute weights sum to one."""
    order = signals.sort_values(ascending=False).index
    half = len(order) // 2
    weights = pd.Series(0.0, index=signals.index)
    weights[order[:half]] = 1.0 / (2 * half)
    weights[order[-half:]] = -1.0 / (2 * half)
    return weights


# ---------------------------------------------------------------------------
# I10: Morgan Stanley Bond Market Indicators
# ---------------------------------------------------------------------------


def cap_signal(z_score):
    """Map a score to -10..+10 with the normal cumulative distribution."""
    capped = 10.0 * (2.0 * norm.cdf(z_score) - 1.0)
    return pd.Series(capped, index=z_score.index) if isinstance(z_score, pd.Series) else capped


def expanding_z(series: pd.Series, min_periods: int = 52) -> pd.Series:
    mean = series.expanding(min_periods).mean()
    std = series.expanding(min_periods).std()
    return (series - mean) / std


def carry_factor(slope_2s10s: pd.Series, realised_vol_1y: pd.Series) -> pd.Series:
    """Curve slope over one-year realised volatility, as an expanding z-score, then capped."""
    return cap_signal(expanding_z(slope_2s10s / realised_vol_1y))


def momentum_factor(excess_index: pd.Series, weeks: int = 17, scale: float = 1.0) -> pd.Series:
    """Index at t-1 less its prior moving average, over the moving average of absolute changes.

    No z-score is used. Choose `scale` so the signal volatility matches the other factors.
    """
    lagged = excess_index.shift(1)
    trend = lagged.shift(1).ewm(span=weeks, adjust=False).mean()
    noise = excess_index.diff().abs().shift(1).ewm(span=weeks, adjust=False).mean()
    return cap_signal(scale * (lagged - trend) / noise)


def equity_factor(local_return_3m: pd.Series, em_minus_dm_3m: pd.Series) -> pd.Series:
    """Weak equities mean long duration, so the sign flips."""
    average = (expanding_z(local_return_3m) + expanding_z(em_minus_dm_3m)) / 2.0
    return cap_signal(-average)


def business_cycle_factor(composite_surprise: pd.Series, scale: float = 1.0) -> pd.Series:
    """A positive surprise means short duration. No z-score is used."""
    return cap_signal(-scale * composite_surprise)


def bmi_combine(factors: pd.DataFrame, dead_band: float = 1.5) -> pd.DataFrame:
    """Combine factor rows (markets in rows, factors in columns) for one date.

    A combined signal inside the band becomes zero. A market keeps its signal only if at
```

```
    least one other market has the same sign. The published UST row is the test case.
    """
    combined = factors.mean(axis=1)
    banded = combined.where(combined.abs() > dead_band, 0.0)
    overall = banded.copy()
    for market in banded.index:
        others = banded.drop(market)
        same_sign = (np.sign(others) == np.sign(banded[market])).any()
        if banded[market] == 0 or not same_sign:
            overall[market] = 0.0
    return pd.DataFrame({"combined": combined, "banded": banded, "overall": overall})


def bmi_cross_market(overall_a: float, overall_b: float) -> float:
    """Difference of two overall signals, when at least one is live."""
    return overall_a - overall_b if (overall_a != 0 or overall_b != 0) else 0.0


# ---------------------------------------------------------------------------
# I13: Deutsche Bank ETF Flow Risk-o-Meter (EFRoM)
# ---------------------------------------------------------------------------

EFROM_RISK_FACTORS = {"US Treasury": -1.0, "Gold": -1.0, "Corp IG": -0.5, "Senior Loans": 0.5, "Corp HY": 0.5, "EM
     Debt": 0.5,
                      "Defensives": 0.5, "Large Cap": 0.5, "Broad Commodities": 1.0, "Domestic Cyclicals": 1.0, "DM
     ": 1.0,
                      "Global Cyclicals": 1.0, "Small Cap": 1.5, "EM": 1.5}


def taarss(cumulative_pct_aum) -> float:
    """Flow trend for one segment and one month: slope through the origin times R-squared.

    The input holds cumulative flow in percent of AUM for trading days 1 to n, with the prior month-end at zero.
    The zero anchor itself is not an input. R-squared comes from an ordinary regression with an intercept.
    """
    y = np.asarray(cumulative_pct_aum, dtype=float)
    x = np.arange(1, len(y) + 1, dtype=float)
    slope = float((x * y).sum() / (x * x).sum())
    r_squared = float(np.corrcoef(x, y)[0, 1] ** 2) if y.std() > 0 else 0.0
    return slope * r_squared


def efrom_level(signals: dict, factors: dict = None) -> float:
    """Risk-weighted sum of raw TAARSS values, not z-scores. Every factor needs a signal."""
    factors = EFROM_RISK_FACTORS if factors is None else factors
    missing = set(factors) - set(signals)
    if missing:
        raise ValueError(f"missing segments: {sorted(missing)}")
    return float(sum(signals[name] * weight for name, weight in factors.items()))
```

A.4 cta.py

```
"""I8: JPMorgan CTA proxy, with the Baz et al. trend signal it cites."""
from __future__ import annotations

import math

import numpy as np
import pandas as pd

PAIRS = ((8, 24), (16, 48), (32, 96))


def half_life(time_scale: float) -> float:
    return math.log(0.5) / math.log(1.0 - 1.0 / time_scale)


def response(y):
    """Position size from trend strength. It peaks near y = sqrt(2), then falls back to zero."""
    return y * np.exp(-(y ** 2) / 4.0) / 0.89


def trend_strength(price: pd.Series, short: int, long: int) -> pd.Series:
```

```
    fast = price.ewm(halflife=half_life(short), adjust=False).mean()
    slow = price.ewm(halflife=half_life(long), adjust=False).mean()
    scaled = (fast - slow) / price.rolling(63, min_periods=63).std()
    return scaled / scaled.rolling(252, min_periods=252).std()


def cta_signal(price: pd.Series) -> pd.Series:
    """Equal-weight average of the response across the three time-scale pairs."""
    return pd.concat([response(trend_strength(price, s, l)) for s, l in PAIRS], axis=1).mean(axis=1, skipna=False)


def tick_rule_imbalance(trades: pd.DataFrame) -> pd.Series:
    """Signed contracts per minute. Columns: time, price, size. A zero tick keeps the last sign.

    The JPMorgan report does not state its classifier. This is a stated default.
    """
    ordered = trades.sort_values("time")
    sign = np.sign(ordered["price"].diff()).replace(0.0, np.nan).ffill().fillna(0.0)
    signed = sign * ordered["size"]
    return signed.groupby(ordered["time"].dt.floor("min")).sum()


def rolling_proxy(imbalance: pd.Series, signal: pd.Series, window: int = 21) -> pd.DataFrame:
    """Rolling correlation and beta between imbalance and signal, one sample per day.

    Pass only the minutes inside the chosen execution windows, summed per day.
    """
    frame = pd.DataFrame({"imbalance": imbalance, "signal": signal}).dropna()
    correlation = frame["imbalance"].rolling(window).corr(frame["signal"])
    beta = frame["imbalance"].rolling(window).cov(frame["signal"]) / frame["signal"].rolling(window).var()
    zero_mean_z = beta / np.sqrt((beta ** 2).expanding(min_periods=window).mean())
    return pd.DataFrame({"correlation": correlation, "beta": beta, "beta_z_zero_mean": zero_mean_z})


def pick_execution_minutes(minute_imbalance: pd.DataFrame, signal: pd.Series, top: int = 5) -> list:
    """Rank minutes of the day by how strongly imbalance follows the signal.

    minute_imbalance has dates in rows and minutes of the day in columns. Calibrate on past data only.
    """
    scores = {minute: minute_imbalance[minute].corr(signal.reindex(minute_imbalance.index))
              for minute in minute_imbalance.columns}
    return [m for m, _ in sorted(scores.items(), key=lambda item: -(item[1] if item[1] == item[1] else -9))[:top]]
```

A.5 data.py

```
"""CFTC feed, canonical field names, timing rules and data checks."""
from __future__ import annotations

import datetime
import json
import time
import urllib.parse
import urllib.request

import numpy as np
import pandas as pd

BASE_URL = "https://publicreporting.cftc.gov/resource/{dataset}.json"

# Six public datasets. Tested live on 2026-11-01.
DATASETS = {
    ("legacy", "futures"): "6dca-aqww",
    ("legacy", "combined"): "jun7-fc8e",
    ("disaggregated", "futures"): "72hh-3qpy",
    ("disaggregated", "combined"): "kh3c-gbw2",
    ("tff", "futures"): "gpe5-46if",
    ("tff", "combined"): "yw9f-hn96",
}

# Canonical name -> API column. The API spelling is not uniform, so keep this table.
COLUMNS = {
    "tff": {
        "open_interest": "open_interest_all",
```

```
        "dealer_long": "dealer_positions_long_all",
        "dealer_short": "dealer_positions_short_all",
        "dealer_spread": "dealer_positions_spread_all",
        "asset_mgr_long": "asset_mgr_positions_long",
        "asset_mgr_short": "asset_mgr_positions_short",
        "asset_mgr_spread": "asset_mgr_positions_spread",
        "lev_money_long": "lev_money_positions_long",
        "lev_money_short": "lev_money_positions_short",
        "lev_money_spread": "lev_money_positions_spread",
        "other_rept_long": "other_rept_positions_long",
        "other_rept_short": "other_rept_positions_short",
        "other_rept_spread": "other_rept_positions_spread",
        "nonrept_long": "nonrept_positions_long_all",
        "nonrept_short": "nonrept_positions_short_all",
    },
    "legacy": {
        "open_interest": "open_interest_all",
        "noncomm_long": "noncomm_positions_long_all",
        "noncomm_short": "noncomm_positions_short_all",
        "noncomm_spread": "noncomm_postions_spread_all",  # the CFTC column really is misspelt
        "comm_long": "comm_positions_long_all",
        "comm_short": "comm_positions_short_all",
        "nonrept_long": "nonrept_positions_long_all",
        "nonrept_short": "nonrept_positions_short_all",
    },
    "disaggregated": {
        "open_interest": "open_interest_all",
        "producer_long": "prod_merc_positions_long",
        "producer_short": "prod_merc_positions_short",
        "swap_long": "swap_positions_long_all",
        "swap_short": "swap__positions_short_all",  # double underscore in the source
        "swap_spread": "swap__positions_spread_all",
        "money_mgr_long": "m_money_positions_long_all",
        "money_mgr_short": "m_money_positions_short_all",
        "money_mgr_spread": "m_money_positions_spread",
        "other_rept_long": "other_rept_positions_long",
        "other_rept_short": "other_rept_positions_short",
        "other_rept_spread": "other_rept_positions_spread",
        "nonrept_long": "nonrept_positions_long_all",
        "nonrept_short": "nonrept_positions_short_all",
    },
}


def fetch_cftc(report: str, code: str, start: str = "1986-01-01", end: str = "2100-01-01",
               basis: str = "futures", retries: int = 3) -> pd.DataFrame:
    """Download one market. Key on the CFTC code, never on the market name."""
    url = BASE_URL.format(dataset=DATASETS[(report, basis)])
    frames, offset, page = [], 0, 5000
    while True:
        query = {"$where": (f"cftc_contract_market_code='{code}' AND "
                            f"report_date_as_yyyy_mm_dd between '{start}T00:00:00.000' "
                            f"and '{end}T00:00:00.000'"),
                 "$order": "report_date_as_yyyy_mm_dd", "$limit": page, "$offset": offset}
        rows = _get_json(url + "?" + urllib.parse.urlencode(query), retries)
        frames.extend(rows)
        if len(rows) < page:
            break
        offset += page
    if not frames:
        raise ValueError(f"no rows for {report}/{basis}/{code}")
    raw = pd.DataFrame(frames)
    out = pd.DataFrame(index=pd.to_datetime(raw["report_date_as_yyyy_mm_dd"]).dt.normalize())
    out.index.name = "report_date"
    for name, column in COLUMNS[report].items():
        if column not in raw:
            raise KeyError(f"{column} missing. The CFTC may have renamed it.")
        out[name] = pd.to_numeric(raw[column].values, errors="coerce")
    out["market_name"] = raw["market_and_exchange_names"].values
    return out[~out.index.duplicated(keep="last")].sort_index()


def _get_json(url: str, retries: int):
    for attempt in range(retries):
```

```
        try:
            with urllib.request.urlopen(url, timeout=60) as response:
                return json.load(response)
        except OSError:
            if attempt == retries - 1:
                raise
            time.sleep(2 ** attempt)
    return []


def publish_time(report_date: pd.Timestamp) -> pd.Timestamp:
    """Normal release: the Friday after the report date, at 15:30 New York time.

    Use this only for history. In production, store the time your job first saw the row.
    """
    days = (4 - report_date.weekday()) % 7
    friday = report_date + datetime.timedelta(days=days)
    return pd.Timestamp(year=friday.year, month=friday.month, day=friday.day, hour=15, minute=30,
                        tz="America/New_York")


def known_at(report: pd.DataFrame, decision_time: pd.Timestamp) -> pd.DataFrame:
    """Return only the rows released at or before the decision time."""
    released = report.index.map(publish_time)
    return report[released <= decision_time]


def check_cftc(report: pd.DataFrame, kind: str = "tff") -> pd.DataFrame:
    """Return one row per failed check. An empty frame means the data passed."""
    problems = []
    gaps = report.index.to_series().diff().dt.days
    for stamp, days in gaps[gaps > 8].items():
        problems.append({"date": stamp, "check": "missing_week", "value": days})
    if kind == "tff":
        long_side = ["dealer_long", "asset_mgr_long", "lev_money_long", "other_rept_long", "nonrept_long"]
        short_side = ["dealer_short", "asset_mgr_short", "lev_money_short", "other_rept_short", "nonrept_short"]
        spreads = ["dealer_spread", "asset_mgr_spread", "lev_money_spread", "other_rept_spread"]
    elif kind == "legacy":
        long_side = ["noncomm_long", "comm_long", "nonrept_long"]
        short_side = ["noncomm_short", "comm_short", "nonrept_short"]
        spreads = ["noncomm_spread"]
    else:
        return pd.DataFrame(problems, columns=["date", "check", "value"])
    for name, columns in (("long_identity", long_side), ("short_identity", short_side)):
        gap = report[columns + spreads].sum(axis=1) - report["open_interest"]
        for stamp, value in gap[gap.abs() > 0.5].items():
            problems.append({"date": stamp, "check": name, "value": value})
    return pd.DataFrame(problems, columns=["date", "check", "value"])


def check_daily(daily: pd.DataFrame, median_rows: int = 60, volume_floor: float = 0.2) -> pd.DataFrame:
    """Flag the daily data faults found in the 2018 workbook."""
    flags = pd.DataFrame(index=daily.index)
    flags["close_outside_range"] = (daily["close"] > daily["high"]) | (daily["close"] < daily["low"])
    flags["high_equals_low"] = daily["high"] == daily["low"]
    flags["missing_range"] = daily["high"].isna() | daily["low"].isna()
    flags["missing_volume"] = daily["volume"].isna()
    typical_volume = daily["volume"].rolling(median_rows, min_periods=10).median()
    flags["volume_collapse"] = daily["volume"] < volume_floor * typical_volume
    flags["duplicate_date"] = daily.index.duplicated()
    return flags


def net(long_side: pd.Series, short_side: pd.Series) -> pd.Series:
    return long_side - short_side


def weekly_returns_to_daily_index(series: pd.Series, index: pd.DatetimeIndex) -> pd.Series:
    """Carry the last weekly value onto a daily index."""
    return series.reindex(index.union(series.index)).ffill().reindex(index)


__all__ = ["fetch_cftc", "publish_time", "known_at", "check_cftc", "check_daily", "net",
           "weekly_returns_to_daily_index", "DATASETS", "COLUMNS", "np"]
```

A.6 stats.py

```
"""Regression and evaluation helpers shared by the indicator tests."""
from __future__ import annotations

import numpy as np
import pandas as pd


def newey_west_ols(y: pd.Series, x: pd.DataFrame, lags: int) -> pd.DataFrame:
    """OLS with Bartlett-kernel Newey-West standard errors. A constant is added."""
    frame = pd.concat([y.rename("__y__"), x], axis=1).dropna()
    design = np.column_stack([np.ones(len(frame)), frame[x.columns].values])
    target = frame["__y__"].values
    xtx_inverse = np.linalg.inv(design.T @ design)
    beta = xtx_inverse @ design.T @ target
    residual = target - design @ beta
    moments = design * residual[:, None]
    meat = moments.T @ moments
    for lag in range(1, lags + 1):
        weight = 1.0 - lag / (lags + 1.0)
        cross = moments[lag:].T @ moments[:-lag]
        meat += weight * (cross + cross.T)
    covariance = xtx_inverse @ meat @ xtx_inverse
    errors = np.sqrt(np.diag(covariance))
    return pd.DataFrame({"beta": beta, "se": errors, "t": beta / errors}, index=["const", *x.columns])


def si_regression(returns_ahead: pd.Series, si: pd.Series, d90=None, d10=None, lags: int | None = None,
                  horizon: int = 1) -> pd.DataFrame:
    """Baseline regression, plus the extreme dummies when they are given."""
    x = pd.DataFrame({"si": si})
    if d90 is not None:
        x["d90"] = d90
        x["d10"] = d10
    return newey_west_ols(returns_ahead, x, lags if lags is not None else horizon)


def forward_change(level: pd.Series, periods: int) -> pd.Series:
    return level.shift(-periods) - level


def reversal_hits(signal: pd.Series, yield_change_ahead: pd.Series) -> dict:
    """signal +1 expects a rally (yield falls). signal -1 expects a sell-off (yield rises)."""
    usable = signal.notna() & yield_change_ahead.notna()
    buys = usable & (signal > 0)
    sells = usable & (signal < 0)
    return {"unconditional_rally": float((yield_change_ahead[usable] < 0).mean()),
            "buy_hit": float((yield_change_ahead[buys] < 0).mean()), "buy_count": int(buys.sum()),
            "sell_hit": float((yield_change_ahead[sells] > 0).mean()), "sell_count": int(sells.sum())}
```

A.7 glgarma.py

```
"""I11: the building blocks of the Yan, Chan and Peters long-memory count model.

The paper samples the posterior with Stan. This module gives the deterministic parts,
so a Stan or NumPyro program can be checked against them.
"""
from __future__ import annotations

import math

import numpy as np


def gegenbauer_psi(d: float, u: float, terms: int) -> np.ndarray:
    """Coefficients of (1 - 2uB + B^2)^(-d). ARFIMA is the special case u = 1."""
    psi = np.zeros(terms)
    psi[0] = 1.0
    if terms > 1:
        psi[1] = 2.0 * d * u
    for j in range(2, terms):
        psi[j] = 2.0 * u * ((d - 1.0) / j + 1.0) * psi[j - 1] - (2.0 * (d - 1.0) / j + 1.0) * psi[j - 2]
```

```
    return psi


def gp_logpmf(y: int, mu: float, nu: float) -> float:
    """Generalised Poisson. The mean is mu and the variance is mu / (1 - nu)^2."""
    lam = mu * (1.0 - nu)
    inner = lam + nu * y
    if lam <= 0 or inner <= 0:
        return -math.inf
    return math.log(lam) + (y - 1) * math.log(inner) - inner - math.lgamma(y + 1)


def od_filter(y: np.ndarray, c: float, d: float, u: float, nu: float = 0.0, terms: int = 100) -> tuple:
    """Observation-driven GLGARMA(0, d, 0) with Pearson residual errors.

    log mu_t = c + sum over j >= 1 of psi_{j-1} * e_{t-j}, where e_t = (y_t - mu_t) / sqrt(mu_t).
    Returns the mean path and the Generalised Poisson log likelihood.
    The recursion can explode for poor parameter draws. It then returns minus infinity.
    """
    psi = gegenbauer_psi(d, u, terms)
    mu = np.zeros(len(y))
    errors = np.zeros(len(y))
    log_likelihood = 0.0
    for t, value in enumerate(y):
        reach = min(t, terms)
        history = sum(psi[j] * errors[t - 1 - j] for j in range(reach))
        if abs(c + history) > 30.0:  # the recursion has blown up, so a sampler must reject this draw
            return mu, -math.inf
        mu[t] = math.exp(c + history)
        errors[t] = (value - mu[t]) / math.sqrt(mu[t])
        log_likelihood += gp_logpmf(int(value), mu[t], nu)
    return mu, log_likelihood


def forecast_scores(actual: np.ndarray, forecast: np.ndarray, history_last: float | None = None) -> dict:
    """MAE, RMSE, MAPE and MASE over the forecast block, as the paper defines them."""
    actual = np.asarray(actual, dtype=float)
    forecast = np.asarray(forecast, dtype=float)
    error = actual - forecast
    naive = np.mean(np.abs(np.diff(actual)))
    return {"mae": float(np.mean(np.abs(error))), "rmse": float(np.sqrt(np.mean(error ** 2))),
            "mape": float(np.mean(np.abs(error / actual)) * 100.0), "mase": float(np.mean(np.abs(error / naive)))}
```

B Contract list and source index

Contract list, `config/contracts.csv`. CFTC units checked live on 1 November 2026.

Symbol	CFTC code	Report for trader groups	Dollars per point
TU	042601	TFF	2,000
FV	044601	TFF	1,000
TY	043602	TFF	1,000
TN	043607	TFF, from 2016	1,000
US	020601	TFF	1,000
WN	020604	TFF, from 2010	1,000
SR3	134741	TFF, from 2018	2,500
ED	132741	TFF, ended 2023	2,500
NG	023651	Disaggregated	10,000

The file also holds the FX futures used by I3 and I7. For those, one point equals the contract size in foreign currency.

Source index. All paths sit under `evidence/`.

Method	Source
P1	`local_disk/MS_Eurodollar_Pain_Trade_Monitor_2014-10-16_page1.pdf`
P2, P4	`gmail/01_pain_monitor_thread_2015/`, June 2015 thread. Its worked example is also in `drive/Pain Monitor Example (local copy).xlsx`.
P3	`code/pain_logic/functions.py`, `code/pain_logic/functions_specification.py`, `code/pain_ui/painMonitor.js`, `gmail/05_portfoliostrat_code_2020/attachments/available_models__a22bb46b.pdf`, pages 8 and 16
P5	`gmail/02_example_workbook_*_2018/attachments/`, also `Example Pain Monitor.xlsx` in the project root. It is the owner's own reading of P1.
I1, I3	`gmail/07_positioning_indicators/db_futures_positioning_2017/`
I2	`gmail/07_positioning_indicators/gs_speculative_positioning_ust_2016/`, `drive/positioning/Global Viewpoint_ 12_17 - Does Speculative Positioning Predict FX_.pdf`
I4, I6	`drive/positioning/Options_implied_position_indicator.pdf`
I5	`drive/positioning/Global Rates Insights_ Filtering CFTC positioning data for a signal.pdf`. DV01: OFR Hedge Fund Monitor TFF dataset, TreasuryDirect security records, FRED yields, and the CME and CBOT conversion factor formula.
I7	`local_disk/214-fx-cookbook-a-recipe-for-systematic-investing-in-currency-markets-1.pdf`
I8	`drive/positioning/JPM_Cross_Asset_CTA_Acti_2023-10-30_4545293.pdf`. Trend formula: Lim, Zohren and Roberts (2019), arXiv 1904.04912, restating Baz, Granger, Harvey, Le Roux and Rattray (2015).
I9	`drive/positioning/CiitiFX+Positioning+Weekly+29Apr19.pdf`
I10	`drive/positioning/MSBondMarketIndicators.pdf`
I11	`gmail/07_positioning_indicators/yan_peters_ust_futures_2023/attachments/SSRN-id2962341__a0670f2d.pdf`
I13	`~/Documents/Flow/`: ETF Flow Risk-o-Meter (EFRoM), `Taars.pdf`, `0900b8c08b403162.pdf`. `~/Documents/Quant Credit/Research Platform/`: Tactical Asset Allocation (TAA) from ETF Flows.
I12	`local_disk/223-investor-positioning-26th-f-feb-1.pdf`, `gmail/07_positioning_indicators/db_rates_positioning_signal_2017/`, `gmail/07_positioning_indicators/ms_oil_positioning_cta_monitor_2016/`, `local_disk/278-robe-roberts-ag-paper-who-does-what-20190702-clearance-nocolor-ada-1.pdf`